

- 3 Procedimiento adaptativo implementado, convergencia $O(M^{-1/2})$ verificada empíricamente y M final justificado por estabilidad numérica.
- 0 No entregado o sin verificación de convergencia.

7. Semana 7. Cadenas de Markov y Sistemas Dinámicos

Objetivo de la cátedra

Formular, analizar y simular sistemas cuya evolución es estocástica y depende únicamente de su estado presente. El estudiante debe ser capaz de construir la matriz de transición (tiempo discreto) o la matriz generadora (tiempo continuo) de un proceso de su propio proyecto, calcular su distribución estacionaria, sus tiempos característicos (recurrencia, primer paso, vida media hasta la falla) y su régimen transitorio, simular trayectorias por métodos exactos, reconocer la transición de fase en procesos espaciales, y comprender cómo el límite determinista de un proceso de Markov produce un sistema dinámico de ecuaciones diferenciales.

Temario

- Procesos estocásticos y propiedad de Markov.
- Matrices de transición: evolución, Chapman–Kolmogorov y potencias.
- Clasificación de estados, distribución estacionaria y teorema ergódico.
- Cadenas absorbentes, tiempos de primer paso y matriz fundamental.
- Cadenas de tiempo continuo: matriz generadora y ecuaciones de Kolmogorov.
- Simulación estocástica exacta: algoritmo de Gillespie.
- Sistemas dinámicos discretos y espaciales: autómatas celulares y percolación (incendios forestales).
- Del proceso estocástico al sistema dinámico determinista: límite de campo medio (modelado epidemiológico).
- Teoría espectral, tiempo de mezcla y caminatas aleatorias (PageRank); sistemas de confiabilidad.
- Interpretación estadística: estimación de la matriz de transición por máxima verosimilitud.

Propósito del tema. La Semana 6 convirtió cada resultado determinista en una distribución de probabilidad, pero esa distribución era *estática*: describía la incertidumbre de un número en un instante. Esta semana introduce el *tiempo* y el *estado*. Un proceso de Markov es la mínima estructura matemática que describe cómo un sistema salta entre configuraciones cuando el futuro depende del presente pero no de la trayectoria

que llevó hasta él. Con tres objetos — la matriz de transición, la distribución estacionaria y los tiempos de primer paso — se responde a preguntas que ningún cálculo determinista puede contestar: ¿qué fracción del tiempo estará un equipo fuera de servicio?, ¿cuánto vivirá en promedio antes de fallar?, ¿cuándo un incendio deja de ser local y cruza todo el dominio?, ¿se extinguirá un brote o se volverá epidemia? La misma maquinaria, llevada al límite, reproduce los sistemas dinámicos deterministas de las semanas anteriores: una cadena de Markov es un sistema dinámico al que se le ha devuelto el azar que la idealización determinista le había quitado.

Resultado de aprendizaje esperado

Al término de la semana el estudiante podrá: (i) identificar si un fenómeno de su proyecto admite una representación markoviana y definir su espacio de estados, su matriz de transición P o su generador Q ; (ii) calcular la distribución estacionaria, la disponibilidad de régimen permanente, los tiempos medios de recurrencia y el tiempo medio hasta la absorción mediante la matriz fundamental; (iii) simular trayectorias del proceso con un método exacto y reproducible en Google Colab, validando los resultados analíticos contra la simulación; (iv) reconocer y cuantificar la transición de fase de un proceso espacial (percolación) y el umbral epidémico de un proceso de propagación; y (v) estimar una matriz de transición a partir de datos observados con su incertidumbre asociada, cerrando el ciclo entre modelo, simulación y evidencia.

7.1. Procesos estocásticos y la propiedad de Markov

Un *proceso estocástico* es una familia de variables aleatorias $\{X_t\}_{t \in \mathcal{T}}$ indexadas por el tiempo, que toman valores en un conjunto $\mathcal{S}$ llamado *espacio de estados*. Cuando $\mathcal{T} = \{0, 1, 2, \dots\}$ el proceso es de *tiempo discreto*; cuando $\mathcal{T} = [0, \infty)$ es de *tiempo continuo*. Cuando $\mathcal{S}$ es finito o numerable, el proceso es de *estados discretos*, que es el caso central de esta semana.

La descripción completa de un proceso general requiere todas sus distribuciones conjuntas, una cantidad de información intratable. La *propiedad de Markov* es la hipótesis de simplificación que vuelve manejable el problema: el futuro depende del pasado únicamente a través del presente. En tiempo discreto,

$$\Pr(X_{n+1} = j \mid X_n = i, X_{n-1} = i_{n-1}, \dots, X_0 = i_0) = \Pr(X_{n+1} = j \mid X_n = i), \quad (191)$$

para todo n y toda historia admisible. Un proceso que cumple (191) es una *cadena de Markov*. La condición no afirma que el sistema carezca de memoria física, sino que toda la memoria relevante está codificada en la definición del estado. Esta es una decisión

de modelado, no una propiedad de la naturaleza: ampliar el estado (por ejemplo, incluir la temperatura previa además de la actual) puede restaurar la propiedad de Markov en sistemas que aparentemente la violan.

La probabilidad de transición de un paso se denota

$$p_{ij} = \Pr(X_{n+1} = j \mid X_n = i), \quad p_{ij} \geq 0, \quad \sum_{j \in \mathcal{S}} p_{ij} = 1. \quad (192)$$

Si p_{ij} no depende de n , la cadena es *homogénea* en el tiempo, hipótesis que se adoptará salvo indicación contraria.

Ejemplo introductorio: el clima de dos estados

El ejemplo canónico más simple tiene dos estados, $\mathcal{S} = \{\text{soleado, lluvioso}\}$. Suponga que tras un día soleado, el siguiente es soleado con probabilidad 0.8 y lluvioso con 0.2; tras un día lluvioso, es soleado con 0.4 y lluvioso con 0.6. La matriz de transición es

$$P = \begin{pmatrix} 0.8 & 0.2 \\ 0.4 & 0.6 \end{pmatrix}.$$

Si hoy está soleado, $\pi_0 = (1, 0)$, la probabilidad de cada clima mañana es $\pi_1 = \pi_0 P = (0.8, 0.2)$, y pasado mañana $\pi_2 = \pi_1 P = (0.72, 0.28)$. Iterando, la distribución converge a $\pi = (2/3, 1/3)$, que satisface $\pi P = \pi$: en el largo plazo, dos tercios de los días son soleados, *independientemente* del clima inicial. Este límite, su existencia y su unicidad son el contenido del teorema ergódico que se formaliza más adelante.

7.2. Matriz de transición: evolución y ecuaciones de Chapman–Kolmogorov

Reuniendo las probabilidades (192) en una matriz $P = [p_{ij}]$ se obtiene la *matriz de transición*, que es *estocástica por filas*: sus entradas son no negativas y cada fila suma uno. El estado de conocimiento sobre el sistema en el paso n se describe mediante un *vector de probabilidad* fila $\pi_n = (\pi_n(1), \pi_n(2), \dots)$, con $\pi_n(i) = \Pr(X_n = i)$. La evolución es una recursión lineal:

$$\pi_{n+1} = \pi_n P, \quad \implies \quad \pi_n = \pi_0 P^n. \quad (193)$$

La probabilidad de transición en n pasos es la entrada (i, j) de la potencia P^n , denotada $p_{ij}^{(n)}$. Estas potencias satisfacen las *ecuaciones de Chapman–Kolmogorov*, que expresan que para ir de i a j en $m + n$ pasos se debe pasar por algún estado intermedio k tras m

pasos:

$$p_{ij}^{(m+n)} = \sum_{k \in S} p_{ik}^{(m)} p_{kj}^{(n)}, \quad \text{es decir} \quad P^{(m+n)} = P^m P^n. \quad (194)$$

La ecuación (194) no es un teorema profundo sino la traducción de la propiedad de Markov a las potencias matriciales; su valor es operativo: reduce el cálculo de cualquier transición multipaso a la multiplicación de matrices, una operación que la Semana 2 ya dejó instalada en el flujo de trabajo computacional. El comportamiento de P^n cuando $n \rightarrow \infty$ contiene toda la información de largo plazo de la cadena y se controla mediante su espectro, como se verá en la teoría espectral.

7.3. Clasificación de estados, distribución estacionaria y teorema ergódico

No toda cadena converge a una distribución única. El comportamiento asintótico depende de propiedades estructurales del grafo de transiciones.

- Un estado j es **accesible** desde i si $p_{ij}^{(n)} > 0$ para algún n . Si i y j son mutuamente accesibles, *comunican*. Una cadena es **irreducible** si todos sus estados comunican (un solo bloque).
- Un estado es **recurrente** si el proceso regresa a él con probabilidad uno, y **transitorio** en caso contrario. En una cadena finita irreducible, todos los estados son recurrentes positivos.
- Un estado es **absorbente** si $p_{ii} = 1$: una vez alcanzado, no se abandona. La “falla” irreparable, la “recuperación” definitiva o el “fin de un brote” son estados absorbentes.
- El **periodo** de un estado es el máximo común divisor de los n para los que $p_{ii}^{(n)} > 0$. Un estado es **aperiódico** si su periodo es 1; basta una autotransición $p_{ii} > 0$ para garantizarlo.

Una distribución π es **estacionaria** (o invariante) si es un punto fijo de la evolución (193):

$$\pi = \pi P, \quad \sum_{i \in S} \pi_i = 1, \quad \pi_i \geq 0. \quad (195)$$

Algebraicamente, π es el vector propio izquierdo de P asociado al autovalor 1, normalizado a suma unitaria. El resultado central de la teoría es el **teorema ergódico**: si una cadena finita es irreducible y aperiódica, entonces existe una única distribución estacionaria π , y

$$\lim_{n \rightarrow \infty} p_{ij}^{(n)} = \pi_j \quad \text{para todo } i, \quad (196)$$

es decir, la cadena “olvida” su condición inicial y la fracción de tiempo que pasa en el estado j tiende a π_j . Una consecuencia directa de gran utilidad práctica es el *tiempo*

medio de recurrencia del estado i , esto es, el número esperado de pasos entre dos visitas consecutivas:

$$m_{ii} = \frac{1}{\pi_i}. \quad (197)$$

La identidad (197) convierte una probabilidad estacionaria en un tiempo físico: si un estado de “fuera de calibración” tiene $\pi = 0.14$, el sistema regresa a él, en promedio, cada 7.25 pasos. Esta es la base del Ejemplo 1.

7.4. Cadenas absorbentes, tiempos de primer paso y matriz fundamental

Cuando la pregunta de ingeniería no es “¿qué fracción del tiempo?” sino “¿cuánto tarda en ocurrir un evento?”, el marco adecuado es el de las *cadenas absorbentes*. Una cadena con estados transitorios y absorbentes se ordena en su *forma canónica*

$$P = \begin{pmatrix} Q & R \\ \mathbf{0} & I \end{pmatrix}, \quad (198)$$

donde Q ($t \times t$) reúne las transiciones entre los t estados transitorios, R las transiciones de transitorios a absorbentes, e I la identidad de los estados absorbentes. La **matriz fundamental**

$$N = (I - Q)^{-1} = \sum_{k=0}^{\infty} Q^k \quad (199)$$

tiene una interpretación directa: N_{ij} es el número esperado de visitas al estado transitorio j partiendo de i , antes de la absorción. De aquí se obtienen dos cantidades de inmenso valor en confiabilidad:

$$\mathbf{t} = N \mathbf{1}, \quad B = N R, \quad (200)$$

donde t_i es el número esperado de pasos hasta la absorción partiendo de i (el *tiempo medio hasta la falla*, MTTF, cuando el estado absorbente es la falla) y B_{ik} es la probabilidad de ser absorbido por el estado k partiendo de i . La serie de Neumann en (199) converge porque el radio espectral de Q es menor que uno: la fuga de probabilidad hacia los estados absorbentes garantiza que $Q^k \rightarrow 0$. El Ejemplo 2 calcula el MTTF de un banco de pruebas por esta vía y lo valida contra simulación Monte Carlo.

7.5. Cadenas de Markov de tiempo continuo: matriz generadora

Muchos fenómenos físicos no avanzan en pasos discretos sino en tiempo continuo: una falla puede ocurrir en cualquier instante. La cadena de Markov de tiempo continuo (CTMC) sustituye las *probabilidades* por paso por *tasas* de transición. La hipótesis markoviana en tiempo continuo equivale a suponer que el tiempo de permanencia en cada

estado es exponencial (la única distribución sin memoria): el sistema permanece en el estado i un tiempo $\sim \text{Exp}(|q_{ii}|)$ y luego salta a j con probabilidad $q_{ij}/|q_{ii}|$.

La dinámica se codifica en la **matriz generadora** $Q = [q_{ij}]$, donde $q_{ij} \geq 0$ ($i \neq j$) es la tasa de transición de i a j , y los elementos diagonales se fijan para que cada fila sume cero:

$$q_{ii} = - \sum_{j \neq i} q_{ij}, \quad \sum_{j \in S} q_{ij} = 0. \quad (201)$$

Las probabilidades de transición $P(t) = [p_{ij}(t)]$ a tiempo t obedecen las *ecuaciones de Kolmogorov*, un sistema lineal de ecuaciones diferenciales:

$$\frac{dP(t)}{dt} = P(t) Q \quad (\text{adelantada}), \quad \frac{dP(t)}{dt} = Q P(t) \quad (\text{atrasada}), \quad (202)$$

cuya solución, con $P(0) = I$, es la *exponencial matricial*

$$P(t) = \exp(Q t) = \sum_{k=0}^{\infty} \frac{(Q t)^k}{k!}. \quad (203)$$

La distribución estacionaria de una CTMC irreducible satisface el análogo continuo de (195):

$$\pi Q = \mathbf{0}, \quad \sum_i \pi_i = 1. \quad (204)$$

El balance de flujos $\pi Q = \mathbf{0}$ expresa que, en régimen permanente, el flujo de probabilidad que entra a cada estado iguala al que sale. El tiempo medio hasta la absorción se obtiene del subgenerador transitorio Q_T (el bloque de Q entre estados no absorbentes):

$$\mathbf{m} = -Q_T^{-1} \mathbf{1}. \quad (205)$$

El Ejemplo 3 modela un cable OPGW como una CTMC de cuatro estados, resuelve su estacionaria por (204), su transitorio por (203) y su MTTF por (205).

7.6. Simulación estocástica exacta: el algoritmo de Gillespie

Resolver (203) entrega la distribución de probabilidad, pero a menudo se requieren *trayectorias individuales*: una realización concreta de cómo evolucionó el sistema. Para una CTMC, el algoritmo de Gillespie (*Stochastic Simulation Algorithm*, SSA) genera trayectorias exactas, no aproximadas. Dado el estado actual, con tasas de evento $a_1, \dots, a_K$ y tasa total $a_0 = \sum_k a_k$:

1. Se genera el tiempo al siguiente evento $\tau \sim \text{Exp}(a_0)$, es decir $\tau = -\ln(r_1)/a_0$ con $r_1 \sim U(0, 1)$.
2. Se selecciona el evento k que ocurre con probabilidad a_k/a_0 , mediante un segundo

número $r_2 \sim U(0, 1)$.

3. Se actualizan el tiempo $t \leftarrow t + \tau$ y el estado según el evento k , y se recalculan las tasas.

El método es exacto porque respeta la estructura exponencial de los tiempos de permanencia y la competencia entre eventos. Su costo crece con el número de eventos, no con el tamaño del paso temporal, lo que lo hace eficiente para poblaciones moderadas. El Ejemplo 5 usa Gillespie para simular un brote epidémico y comparar sus trayectorias estocásticas con el sistema dinámico determinista.

7.7. Markov en el espacio y en grafos: autómatas celulares, percolación y teoría espectral

7.7.1. Campos de Markov espaciales y percolación

Cuando el estado no es un escalar sino una configuración espacial — una retícula donde cada celda tiene un estado que depende de sus vecinas — el proceso es un *campo de Markov* y su evolución temporal es un *autómata celular probabilístico*. El ejemplo paradigmático es la propagación: una celda “activa” (ardiendo, infectada, dañada) activa a cada vecina susceptible con probabilidad p . Estos modelos exhiben un fenómeno que ningún sistema lineal posee: una *transición de fase*. Existe un valor crítico p_c tal que para $p < p_c$ la activación permanece localizada y se extingue, mientras que para $p > p_c$ aparece, con probabilidad positiva, un *cúmulo percolante* que cruza todo el dominio. Para la propagación a cuatro vecinos en una red cuadrada, el problema es equivalente a la percolación de enlaces, cuyo umbral exacto es

$$p_c = \frac{1}{2}. \quad (206)$$

El parámetro de orden (la fracción activada, o la probabilidad de que la activación alcance el borde) cambia de forma abrupta alrededor de p_c , con fluctuaciones máximas justo en el umbral. El Ejemplo 4 reproduce esta transición para un incendio forestal y estima p_c empíricamente.

7.7.2. Caminatas aleatorias en grafos, teoría espectral y mezcla

Una cadena de Markov sobre los nodos de un grafo dirigido es una *caminata aleatoria*: en cada paso, el caminante salta a un nodo vecino. Su distribución estacionaria mide la “importancia” de cada nodo, definición que está en el corazón del algoritmo PageRank. La velocidad con que la cadena alcanza su estacionaria la determina el *espectro* de P . El autovalor dominante de una matriz estocástica irreducible es siempre $\lambda_1 = 1$ (teorema de Perron–Frobenius), con vector propio izquierdo π . El segundo autovalor en magnitud,

$|\lambda_2| < 1$, controla la convergencia:

$$\|\pi_n - \pi\|_{TV} \leq C |\lambda_2|^n, \quad \text{brecha espectral} = 1 - |\lambda_2|. \quad (207)$$

Una brecha espectral grande implica convergencia rápida (mezcla veloz); una brecha pequeña, convergencia lenta. El *tiempo de mezcla* t_{mix} es el número de pasos para que la distancia en variación total caiga por debajo de un umbral, y escala como $t_{\text{mix}} \sim 1/(1 - |\lambda_2|)$. El Ejemplo 6 explota esta teoría: la matriz de Google garantiza $|\lambda_2| \leq \alpha$, lo que asegura una tasa de convergencia α^n de la iteración de potencias, independiente del tamaño del grafo.

7.8. Del proceso estocástico al sistema dinámico determinista

El vínculo conceptual más importante de la semana es que un sistema dinámico determinista (como los de las Semanas 4 y 5) es el *límite de campo medio* de un proceso de Markov cuando el número de individuos crece. Considérese un proceso con poblaciones $\mathbf{X} = (X_1, \dots, X_m)$ y eventos que las modifican con tasas proporcionales a la población. Al escalar por el tamaño total N y definir las fracciones $\mathbf{x} = \mathbf{X}/N$, la ley de los grandes números para procesos de salto (teorema de Kurtz) establece que, cuando $N \rightarrow \infty$, las trayectorias estocásticas convergen a la solución del sistema de ecuaciones diferenciales

$$\frac{d\mathbf{x}}{dt} = \sum_k \nu_k f_k(\mathbf{x}), \quad (208)$$

donde ν_k es el cambio de estado del evento k y f_k su tasa por unidad de población. Para el modelo epidemiológico SIR, con tasa de contagio β y de recuperación γ , este límite produce el sistema dinámico clásico

$$\frac{dS}{dt} = -\beta \frac{SI}{N}, \quad \frac{dI}{dt} = \beta \frac{SI}{N} - \gamma I, \quad \frac{dR}{dt} = \gamma I, \quad (209)$$

cuyo comportamiento queda determinado por el *número reproductivo básico*

$$R_0 = \frac{\beta}{\gamma}. \quad (210)$$

Si $R_0 > 1$ la infección crece inicialmente (umbral epidémico $S^* = N/R_0$); si $R_0 < 1$ se extingue. La descripción determinista (209), sin embargo, oculta un fenómeno puramente estocástico: aun con $R_0 > 1$, un brote iniciado por pocos infectados puede extinguirse por azar antes de despegar. La teoría de procesos de ramificación estima esta probabilidad

de *brote menor* como $(1/R_0)^{I_0}$, de modo que la probabilidad de brote mayor es

$$P_{\text{mayor}} \approx 1 - \left(\frac{1}{R_0} \right)^{I_0}. \quad (211)$$

El Ejemplo 5 compara (209), (210) y (211) contra el ensamble estocástico, mostrando que el modelo determinista describe bien el “brote mayor” pero es ciego a la extinción temprana.

7.9. Interpretación estadística: estimación de la matriz de transición

Cerrando el ciclo entre modelo y datos, una matriz de transición rara vez se conoce a priori: debe estimarse a partir de una trayectoria observada. Si n_{ij} es el número de transiciones $i \rightarrow j$ contadas en los datos y $n_i = \sum_j n_{ij}$ el total de salidas de i , el estimador de *máxima verosimilitud* es la frecuencia relativa

$$\hat{p}_{ij} = \frac{n_{ij}}{n_i}, \quad \widehat{\text{se}}(\hat{p}_{ij}) = \sqrt{\frac{\hat{p}_{ij}(1 - \hat{p}_{ij})}{n_i}}, \quad (212)$$

con error estándar de Wald asintótico. Cada fila de P se estima de forma independiente como una distribución multinomial, y la incertidumbre de cada $\hat{p}_{ij}$ se reduce como $n_i^{-1/2}$: estimar bien las transiciones de un estado raro exige observarlo muchas veces. La ecuación (212) conecta esta semana con la metrología de la Semana 6: la matriz de transición es ella misma un mensurando con incertidumbre, y propagar esa incertidumbre a la distribución estacionaria o al MTTF es un problema de propagación que las técnicas Monte Carlo de la semana anterior resuelven directamente.

7.10. Ejemplo 1 — Deriva de calibración de un sensor: la cadena de dos estados

7.10.1. Planteamiento

Un sensor del sistema de calibración del proyecto de Álvarez Sánchez se inspecciona periódicamente (cada semana) y se clasifica en uno de dos estados: *en calibración* (dentro del error máximo permisible) o *fuera de calibración*. Entre inspecciones, el sensor puede derivar fuera de tolerancia con probabilidad a por semana; cuando se detecta fuera de tolerancia, se recalibra y vuelve a tolerancia con probabilidad b . El proceso es una cadena de Markov de dos estados con matriz de transición

$$P = \begin{pmatrix} 1-a & a \\ b & 1-b \end{pmatrix} = \begin{pmatrix} 0.92 & 0.08 \\ 0.50 & 0.50 \end{pmatrix}, \quad (213)$$

con $a = 0.08$ y $b = 0.50$. La pregunta metrológica no es “¿está calibrado hoy?” sino “¿qué fracción del tiempo opera fuera de tolerancia?” y “¿cada cuánto debe programarse la recalibración?”. Los parámetros se resumen en la Tabla 34.

Tabla 34: Parámetros de la cadena de deriva de calibración (Ec. 213).

Magnitud	Valor	Interpretación
Prob. de deriva a	0.08	$\Pr(\text{en} \rightarrow \text{fuera})$ por semana
Prob. de recalibración b	0.50	$\Pr(\text{fuera} \rightarrow \text{en})$ por semana
Paso temporal	1 semana	periodo de inspección

7.10.2. Resultados y lectura metodológica

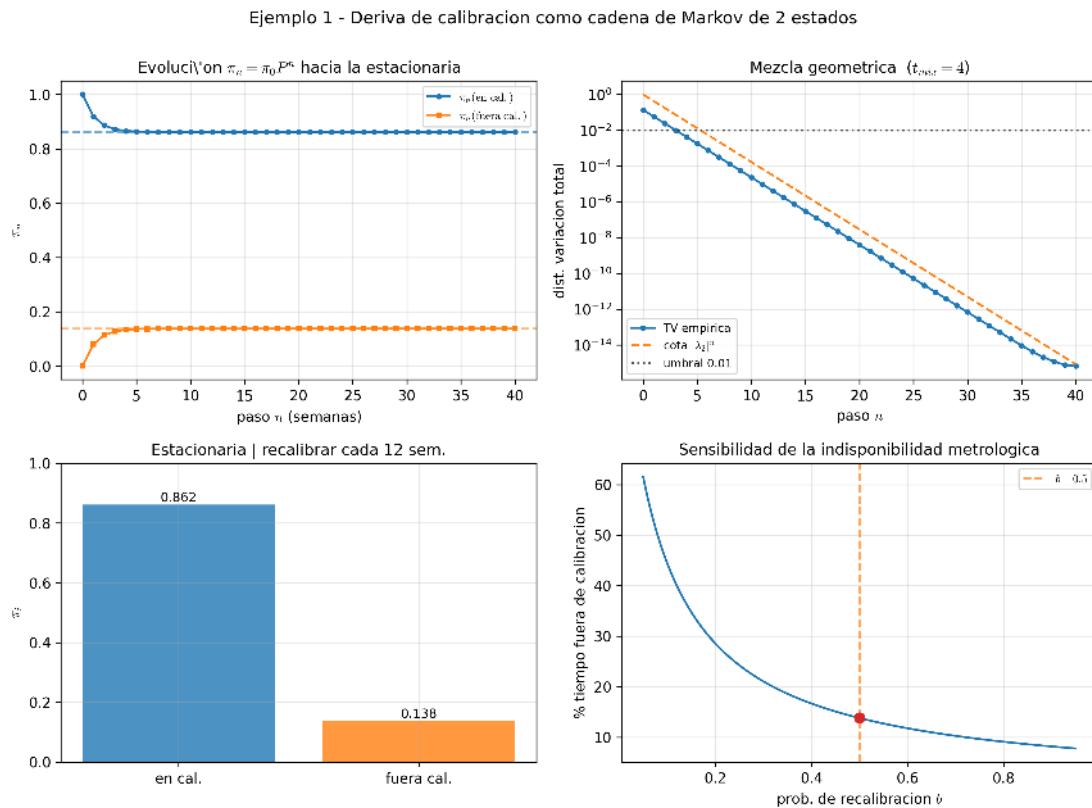


Figura 49: Deriva de calibración como cadena de Markov de dos estados. Panel 1: evolución de la distribución $\pi_n = \pi_0 P^n$ (Ec. 193) hacia la estacionaria. Panel 2: convergencia geométrica en distancia de variación total y su cota $|\lambda_2|^n$ (Ec. 207). Panel 3: distribución estacionaria e intervalo de recalibración recomendado. Panel 4: sensibilidad de la indisponibilidad metrológica a la probabilidad de recalibración b . Resultados del Código 1 del Anexo de esta semana.

1. **La distribución estacionaria cuantifica la indisponibilidad metrológica.** La solución de (195) es $\pi = (b, a)/(a + b) = (0.862, 0.138)$. El sensor opera *fuera de*

tolerancia el 13.79 % del tiempo en régimen permanente, una cifra que ningún análisis de un solo punto de calibración podría revelar. Esta es la indisponibilidad metrológica del instrumento bajo la política de mantenimiento actual.

2. **Los tiempos de primer paso fijan el intervalo de recalibración.** El tiempo medio de deriva en $\rightarrow$ fuera es $1/a = 12.5$ semanas: en promedio, el sensor tarda doce semanas y media en salir de tolerancia tras una calibración. Programar la recalibración *preventiva* en torno a ese horizonte, en lugar de esperar la detección *correctiva*, mantiene el instrumento dentro de tolerancia la mayor parte de su vida útil. El tiempo medio de recurrencia del estado “en calibración” es $1/\pi_0 = 1.16$ semanas (Ec. 197).
3. **La brecha espectral mide la rapidez del olvido.** Los autovalores de P son 1 y $1 - a - b = 0.42$; la brecha espectral es $a + b = 0.58$ y la cadena alcanza su estacionaria (variación total < 0.01) en apenas $t_{\text{mix}} = 4$ pasos. El sistema olvida su condición inicial con rapidez: el estado de calibración de hace un mes es irrelevante para predecir el de hoy.
4. **La interpretación estadística cierra el ciclo.** Simulando una trayectoria de 5000 semanas y aplicando el estimador de máxima verosimilitud (212) se recupera $\hat{P} = \begin{pmatrix} 0.913 & 0.087 \\ 0.512 & 0.488 \end{pmatrix}$, con errores estándar de Wald de 0.004 y 0.019 respectivamente. La fila del estado raro (“fuera”) se estima con cuatro veces más incertidumbre porque se visita cuatro veces menos: la calidad del modelo está limitada por la frecuencia de observación de cada estado.

7.11. Ejemplo 2 — Confiabilidad de un banco de pruebas: cadena con reparación

7.11.1. Planteamiento

El banco de pruebas de bomba centrífuga del proyecto de Leyva Jiménez y Zavala Morales opera diariamente y puede encontrarse en tres estados: *operativo* (nominal), *degradado* (vibración o cavitación incipiente, aún funcional) y en *falla* (fuera de servicio). Las transiciones diarias, incluyendo la reparación, se modelan con la matriz

$$P = \begin{pmatrix} 0.95 & 0.04 & 0.01 \\ 0.30 & 0.65 & 0.05 \\ 0.50 & 0.00 & 0.50 \end{pmatrix}, \quad (214)$$

en el orden (O, D, F). La cadena es irreducible (todo estado comunica gracias a la reparación) y aperiódica, por lo que posee estacionaria única. Las preguntas de ingeniería son la *disponibilidad* de régimen permanente y el *tiempo medio hasta la primera falla* (MTTF) partiendo de un equipo nuevo. Los parámetros aparecen en la Tabla 35.

Tabla 35: Transiciones diarias del banco de pruebas (Ec. 214).

Desde \ a	Operat.	Degrad.	Falla	Mecanismo
Operativo	0.95	0.04	0.01	desgaste / falla súbita
Degradado	0.30	0.65	0.05	reparación / progresión
Falla	0.50	0.00	0.50	reparación a operativo

7.11.2. Disponibilidad y MTTF

La estacionaria, solución de (195), es $\pi = (0.873, 0.0998, 0.0274)$. Para el MTTF se trata la falla como absorbente y se usa la matriz fundamental (199) sobre el bloque transitorio Q entre $\{O, D\}$, obteniendo el tiempo esperado a la absorción (200).

7.11.3. Resultados y lectura metodológica

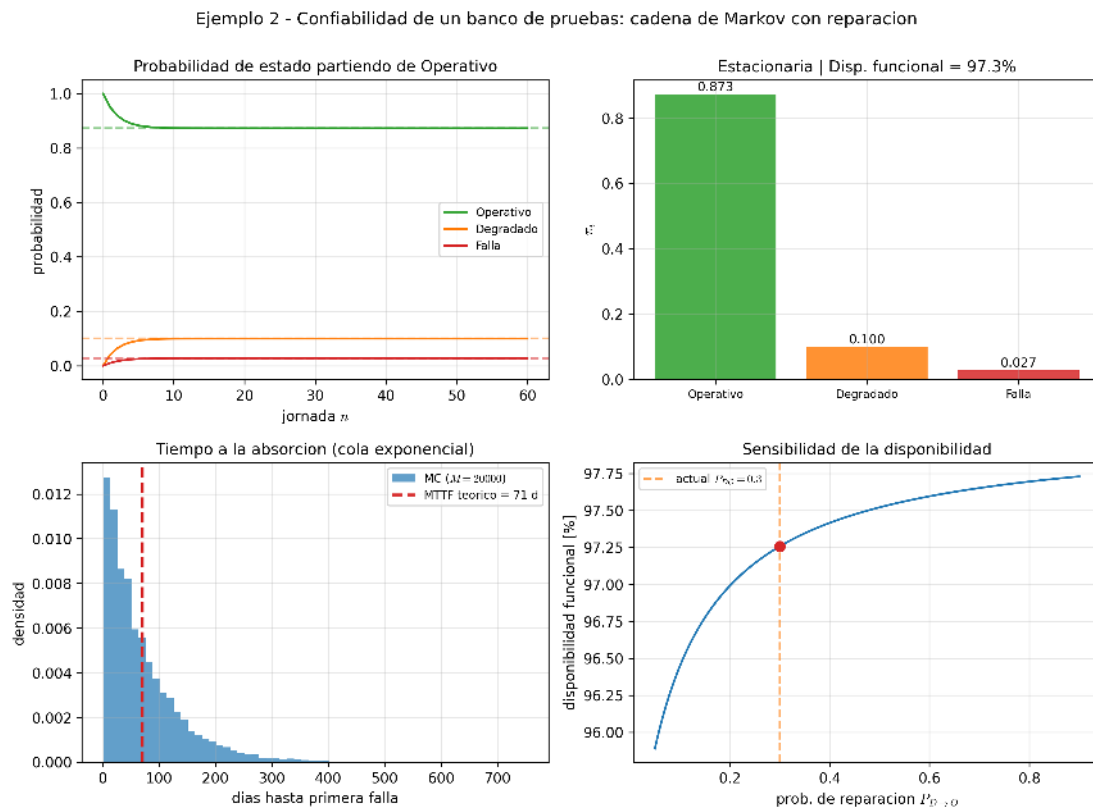


Figura 50: Confiabilidad del banco de pruebas. Panel 1: probabilidad transitoria de cada estado partiendo de Operativo (Ec. 193). Panel 2: distribución estacionaria y disponibilidad funcional. Panel 3: histograma Monte Carlo del tiempo a la primera falla con el MTTF teórico de la matriz fundamental (Ec. 200). Panel 4: sensibilidad de la disponibilidad a la probabilidad de reparación $P_{D \rightarrow O}$. Resultados del Código 2 del Anexo de esta semana.

1. **Disponibilidad de régimen permanente.** La disponibilidad estricta (solo Operativo) es $\pi_O = 87.28\%$ y la *funcional* (Operativo o Degradado, ambos productivos) es $\pi_O + \pi_D = 97.26\%$. La indisponibilidad por falla es $\pi_F = 2.74\%$. La distinción entre disponibilidad estricta y funcional es una decisión de criterio: depende de si el estado degradado permite ensayos válidos o no.
2. **El MTTF se obtiene exactamente y se valida por simulación.** La matriz fundamental da un tiempo medio a la primera falla de 70.91 días partiendo de Operativo. La simulación Monte Carlo independiente ($M = 20000$ trayectorias) arroja 70.86 ± 0.49 días, coincidiendo dentro del error estándar: el cálculo algebraico y la simulación se validan mutuamente. El histograma del tiempo a la falla exhibe la cola aproximadamente exponencial característica de los tiempos de absorción.
3. **El MTBF de régimen permanente difiere del MTTF.** El flujo estacionario hacia la falla, $\sum_i \pi_i p_{iF}$, da un tiempo medio entre fallas de 36.45 días en operación continua. Es menor que el MTTF de 70.91 días porque, en régimen, una fracción del tiempo el equipo ya está degradado y por tanto más cerca de la falla que un equipo nuevo: la condición inicial importa para el MTTF pero no para el MTBF.
4. **La sensibilidad guía la inversión en mantenimiento.** El Panel 4 muestra que aumentar la probabilidad de reparación del estado degradado eleva la disponibilidad de forma monótona pero con rendimiento decreciente. Saber dónde se aplanan la curva indica hasta qué punto invertir en mantenimiento preventivo del estado degradado mejora la disponibilidad antes de toparse con el límite impuesto por las fallas súbitas desde el estado operativo.

7.12. Ejemplo 3 — Confiabilidad de un OPGW: cadena de Markov de tiempo continuo

7.12.1. Planteamiento

El cable OPGW del proyecto de Méndez Gutiérrez y Jurado Pérez está sujeto a sollicitaciones que ocurren en tiempo continuo, no en pasos discretos. Se modela como una CTMC de cuatro estados: *normal* (N), *sobrecarga térmica* (T, corriente elevada, reversible por enfriamiento), *hielo/galloping* (H, sobrecarga mecánica, reversible) y *falla* (F, rotura o cortocircuito que requiere reparación). Las tasas de transición (eventos por año) definen la matriz generadora Q según (201); sus valores aparecen en la Tabla 36. La sobrecarga térmica conecta este ejemplo con el modelo IEC 60949 de la Semana 6: allí se cuantificó la incertidumbre de la temperatura de cortocircuito; aquí se cuantifica con qué frecuencia y por cuánto tiempo el cable opera en ese régimen a lo largo de su vida.

Tabla 36: Tasas de transición del modelo CTMC del OPGW (1/año), usadas para construir el generador Q (Ec. 201).

Transición	Tasa [1/año]	Tiempo medio
Normal $\rightarrow$ sobrecarga térmica	2.0	6 meses
Sobrecarga térmica $\rightarrow$ normal	50.0	7.3 días
Sobrecarga térmica $\rightarrow$ falla	0.5	2 años
Normal $\rightarrow$ hielo/galloping	1.0	1 año
Hielo $\rightarrow$ normal	20.0	18.3 días
Hielo $\rightarrow$ falla	0.3	3.3 años
Falla $\rightarrow$ normal (reparación)	12.0	1 mes

7.12.2. Resultados y lectura metodológica

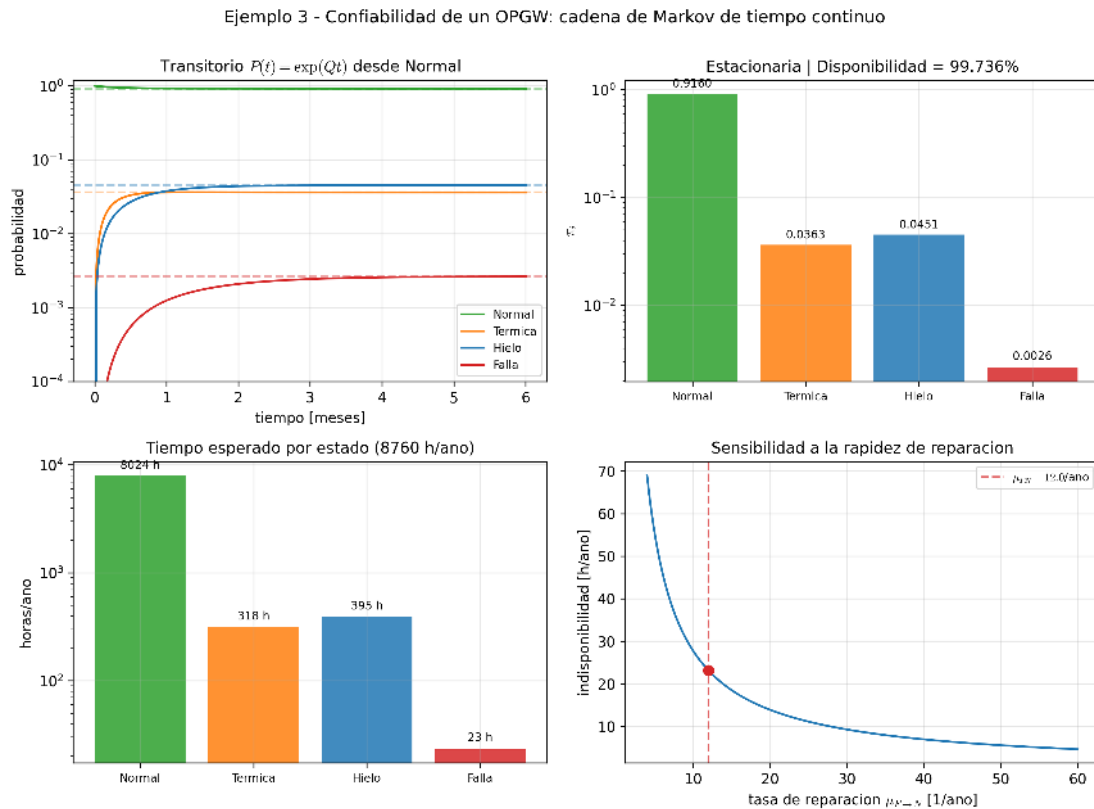


Figura 51: Confiabilidad del OPGW como CTMC. Panel 1: probabilidades transitorias $P(t) = \exp(Qt)$ partiendo de Normal (Ec. 203). Panel 2: distribución estacionaria (escala logarítmica) y disponibilidad. Panel 3: horas esperadas por año en cada estado. Panel 4: sensibilidad de la indisponibilidad a la tasa de reparación $\mu_{F \rightarrow N}$. Resultados del Código 3 del Anexo de esta semana.

1. **La estacionaria entrega la disponibilidad del activo.** La solución de (204) es $\pi = (0.916, 0.0363, 0.0451, 0.00264)$. La disponibilidad (todo estado salvo falla) es 99.736 % y la indisponibilidad por falla, 0.264 %, equivale a 23.12 horas de fuera

de servicio por año. Esta es la cifra que un operador de red eléctrica necesita para sus índices de continuidad de servicio.

2. **El tiempo en cada estado se traduce a horas físicas.** Multiplicando π por las 8760 horas del año, el cable pasa 317.8 h/año en sobrecarga térmica y 395.3 h/año en condición de hielo. La fracción de tiempo en sobrecarga térmica (3.63 %) es precisamente la exposición acumulada al régimen que el modelo térmico de la Semana 6 analizó puntualmente: el envejecimiento del material es función de esta exposición integrada, no de un único evento de cortocircuito.
3. **El MTTF se obtiene del subgenerador transitorio.** Tratando la falla como absorbente, (205) da un tiempo medio hasta la primera falla de 31.49 años partiendo del estado normal. Este horizonte, comparado con la vida de diseño del cable, indica si el activo está sobredimensionado o si requiere inspección reforzada.
4. **La rapidez de reparación gobierna la indisponibilidad.** El Panel 4 muestra que la indisponibilidad anual decae al aumentar la tasa de reparación $\mu_{F \rightarrow N}$ (equivalente a reducir el tiempo medio de reparación, MTTR). La relación es fuertemente no lineal: pasar de un MTTR de tres meses a uno de un mes recorta las horas de indisponibilidad de forma desproporcionada, lo que prioriza la logística de reparación rápida sobre otras inversiones.

7.13. Ejemplo 4 — Incendio forestal: autómatas celulares probabilísticos y percolación

7.13.1. Planteamiento

El proyecto de monitoreo aéreo de García Ojeda admite una extensión natural a la vigilancia de incendios forestales mediante UAV. El fenómeno se modela como un autómata celular probabilístico sobre una retícula $L \times L$: cada celda contiene un árbol con densidad d ; una celda ardiendo propaga el fuego a cada vecina verde (cuatro vecinos) con probabilidad p y luego se consume. El proceso es un campo de Markov espacial cuyo interés reside en su *transición de fase*: para densidad $d = 1$, el modelo equivale a percolación de enlaces en la red cuadrada, con umbral crítico exacto $p_c = 1/2$ (Ec. 206). Desde el monitoreo por UAV, los parámetros d (densidad de combustible) y p (probabilidad de propagación, función de viento y humedad) se estiman de la imagerie aérea, y el modelo predice la probabilidad de un gran incendio. Los parámetros se listan en la Tabla 37.

Tabla 37: Parámetros del autómata celular de incendio forestal.

Parámetro	Valor	Significado
Tamaño de retícula L	80 (barrido), 120 (demo)	dominio cuadrado $L \times L$
Densidad de árboles d	1.0	fracción de celdas con combustible
Prob. de propagación p	barrido [0.30, 0.75]	ignición por vecino
Réplicas por valor de p	60	promedios de ensamble
Ignición	celda central	foco inicial único

7.13.2. Resultados y lectura metodológica

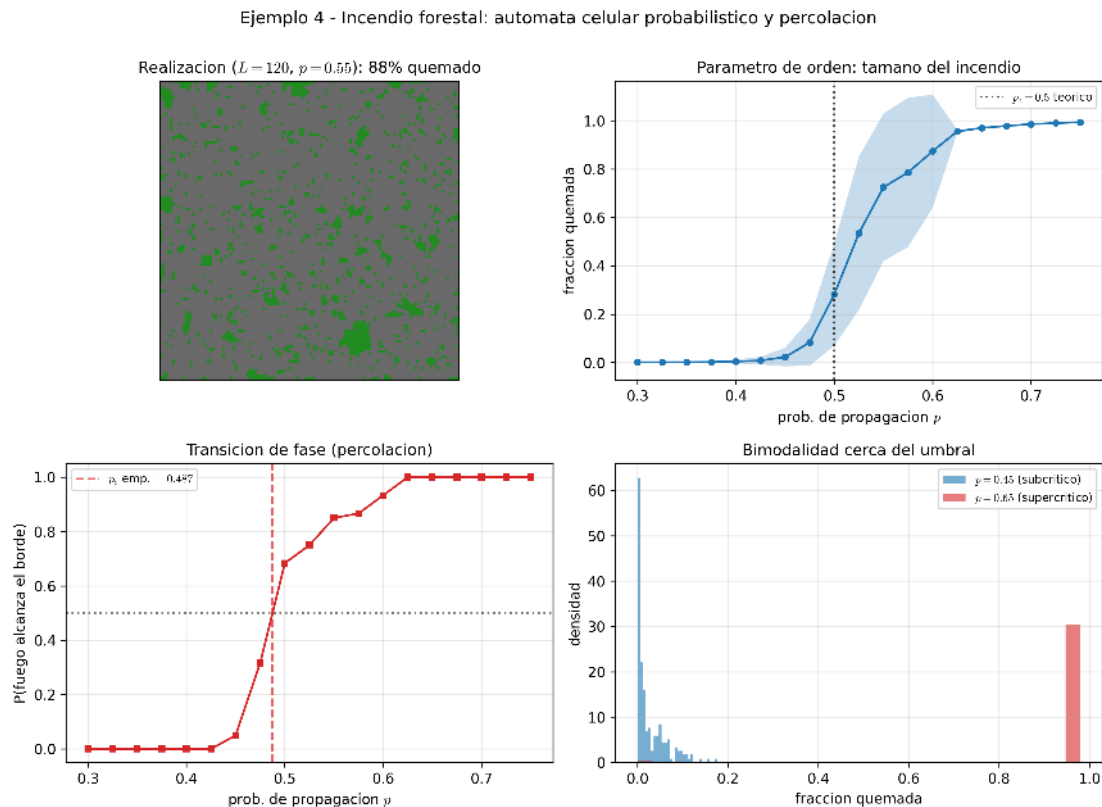


Figura 52: Incendio forestal como autómata celular probabilístico. Panel 1: una realización ($L = 120$, $p = 0.55$) con celdas verdes, quemadas y vacías. Panel 2: fracción quemada media frente a p con banda de dispersión y umbral teórico $p_c = 0.5$ (Ec. 206). Panel 3: probabilidad de que el fuego alcance el borde (parámetro de orden de percolación) con el p_c empírico. Panel 4: bimodalidad de la fracción quemada en regímenes subcrítico y supercrítico. Resultados del Código 4 del Anexo de esta semana.

1. **Existe un umbral crítico, no una transición suave.** La fracción quemada media salta de 0.027 en $p = 0.45$ (subcrítico) a 0.960 en $p = 0.65$ (supercrítico). La

probabilidad de que el fuego cruce el dominio cruza el 50 % en un p_c empírico de 0.4875, en excelente acuerdo con el valor exacto $p_c = 1/2$ de la percolación de enlaces. Una variación pequeña de p alrededor del umbral cambia cualitativamente el desenlace: este es el rasgo que distingue a un sistema crítico de uno lineal.

2. **La gestión del riesgo se concentra en el umbral.** La consecuencia operativa para el monitoreo por UAV es que la prioridad no es estimar p con precisión uniforme, sino determinar de qué lado del umbral p_c se encuentra el bosque bajo las condiciones meteorológicas actuales. Una reducción de combustible (bajar d) o de propagación (cortafuegos, que reduce p efectivo) tiene un retorno desproporcionado cuando empuja al sistema de supercrítico a subcrítico.
3. **La incertidumbre es máxima en la criticidad.** El Panel 4 muestra que lejos del umbral el resultado es casi determinista (fuego local o fuego total), pero cerca de p_c la fracción quemada es altamente variable de una realización a otra. La predicción de un incendio individual es intrínsecamente incierta en el régimen crítico, lo que obliga a razonar en términos de probabilidad de gran incendio y no de un pronóstico puntual.

7.14. Ejemplo 5 — Modelo epidemiológico estocástico: del SIR de Markov al sistema dinámico

7.14.1. Planteamiento

El modelo SIR es a la vez una cadena de Markov de tiempo continuo y, en su límite, un sistema dinámico de ecuaciones diferenciales; por ello sirve de puente conceptual entre esta semana y las Semanas 4–5. Una población de $N = 1000$ individuos se divide en susceptibles (S), infectados (I) y recuperados (R). Dos eventos dirigen la dinámica: contagio $S + I \rightarrow 2I$ con tasa $\beta SI/N$, y recuperación $I \rightarrow R$ con tasa γI . Con $\beta = 0.60$ y $\gamma = 0.25$ se obtiene un número reproductivo básico $R_0 = \beta/\gamma = 2.40$. Se simula el proceso estocástico exacto con Gillespie y se compara con la solución determinista (209). Los parámetros aparecen en la Tabla 38.

Tabla 38: Parámetros del modelo epidemiológico SIR (Ec. 209 y Ec. 210).

Parámetro	Valor	Significado
Población N	1000	tamaño total constante
Infectados iniciales I_0	5	semilla del brote
Tasa de contagio β	0.60 d^{-1}	contactos efectivos
Tasa de recuperación γ	0.25 d^{-1}	inverso del periodo infeccioso
Número reproductivo R_0	2.40	β/γ
Realizaciones Gillespie M	3000	ensamble estocástico

7.14.2. Resultados y lectura metodológica

Ejemplo 5 - SIR estocástico (Gillespie) y su límite determinista

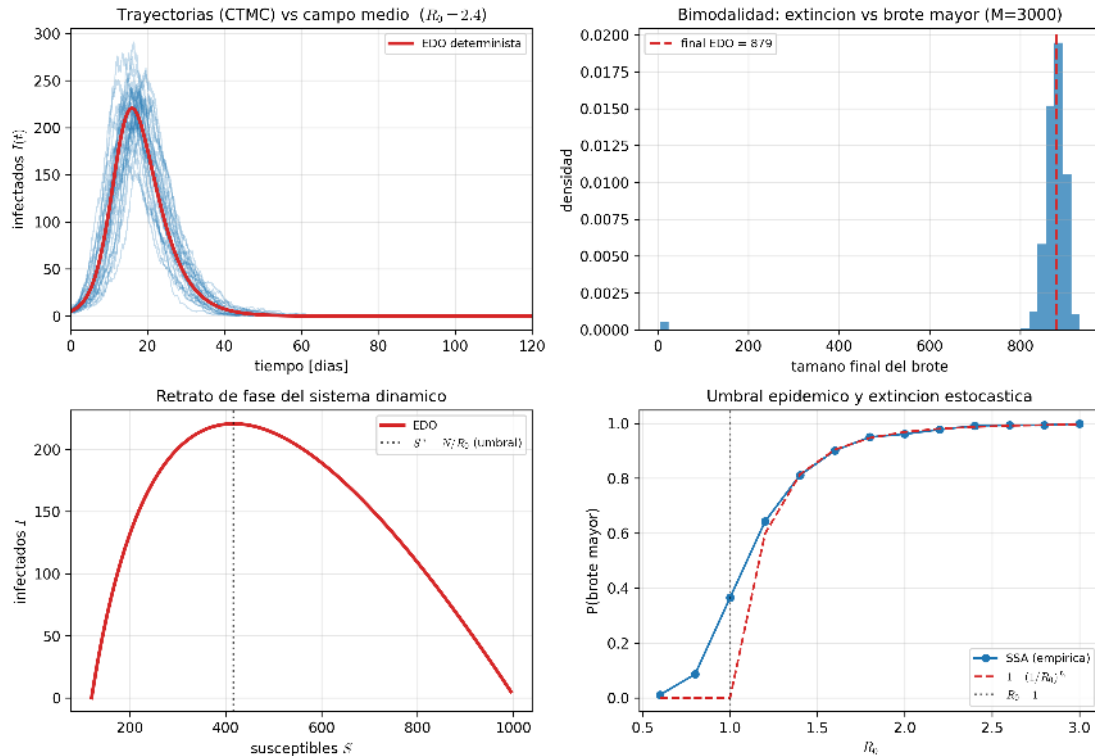


Figura 53: SIR estocástico (Gillespie) y su límite determinista. Panel 1: trayectorias estocásticas $I(t)$ y solución de la EDO (Ec. 209). Panel 2: distribución del tamaño final del brote, bimodal (extinción vs brote mayor). Panel 3: retrato de fase S - I del sistema dinámico con el umbral $S^* = N/R_0$. Panel 4: probabilidad de brote mayor frente a R_0 , comparada con la teoría $1 - (1/R_0)^{I_0}$ (Ec. 211). Resultados del Código 5 del Anexo de esta semana.

1. El determinista describe el brote mayor; el estocástico, también la extinción.

La EDO predice un pico de $I_{\text{máx}} = 220.6$ infectados en $t = 16.0$ días y un tamaño final de 879.5 (87.9 % de la población). El ensamble estocástico, condicionado a brote mayor, da un tamaño final medio de 878.1 (87.8 %): el límite de campo medio es preciso *cuando el brote despegue*. Pero el modelo determinista no puede representar la rama de extinción temprana, que sí aparece en la distribución bimodal del Panel 2.

2. La extinción estocástica es un fenómeno de número pequeño. La probabilidad empírica de brote mayor es 0.990, en acuerdo con la predicción de procesos de ramificación $1 - (1/R_0)^{I_0} = 0.987$ (Ec. 211). Con solo $I_0 = 5$ infectados iniciales y $R_0 = 2.4$, alrededor del 1 % de los brotes se extingue por azar pese a tener $R_0 > 1$. Este efecto sería invisible en un modelo puramente determinista y es decisivo en las fases iniciales de una epidemia o en la introducción de una especie.

3. **El umbral $R_0 = 1$ es una transición de fase dinámica.** El Panel 4 muestra que la probabilidad de brote mayor es prácticamente nula para $R_0 < 1$ y crece de forma abrupta al superar la unidad, replicando en el plano dinámico la misma estructura de transición que la percolación del Ejemplo 4. El umbral $S^* = N/R_0$ del retrato de fase marca el punto donde $\dot{I} = 0$: a la izquierda la epidemia decae, a la derecha crece.
4. **Mismo proceso, dos descripciones complementarias.** El valor pedagógico del ejemplo es mostrar que la EDO (209) no es un modelo distinto del estocástico, sino su límite de campo medio (Ec. 208). Elegir entre uno y otro es una decisión de escala: poblaciones grandes y preguntas promedio favorecen la EDO; poblaciones pequeñas, eventos raros y probabilidad de extinción exigen el proceso de Markov.

7.15. Ejemplo 6 — Ranking por caminata aleatoria (PageRank) para recuperación en RAG

7.15.1. Planteamiento

El sistema RAG (*Retrieval-Augmented Generation*) industrial del proyecto de Rodríguez Morales debe ordenar documentos por relevancia. Cuando los documentos forman un grafo dirigido (por citación, enlace o similitud semántica umbralizada), su importancia puede medirse como la distribución estacionaria de una caminata aleatoria sobre el grafo: el algoritmo PageRank. El navegante aleatorio sigue un enlace saliente con probabilidad α (*damping*) y “teletransporta” a un documento al azar con probabilidad $1 - \alpha$; los nodos sin enlaces salientes (*colgantes*) se tratan como teletransporte total. Esto define la matriz de Google

$$G = \alpha S + (1 - \alpha) \frac{1}{n} \mathbf{1}\mathbf{1}^\top, \quad (215)$$

con S la matriz de transición por filas y $\alpha = 0.85$. El vector PageRank es el vector propio izquierdo de G con autovalor 1 (Perron–Frobenius), calculado por iteración de potencias. Los parámetros aparecen en la Tabla 39.

Tabla 39: Parámetros del modelo PageRank sobre el grafo de documentos (Ec. 215).

Parámetro	Valor	Significado
Número de documentos n	14	nodos del grafo
Damping α	0.85	prob. de seguir un enlace
Nodos colgantes	1	sin enlaces salientes
Tolerancia de convergencia	10^{-15}	criterio de parada

7.15.2. Resultados y lectura metodológica

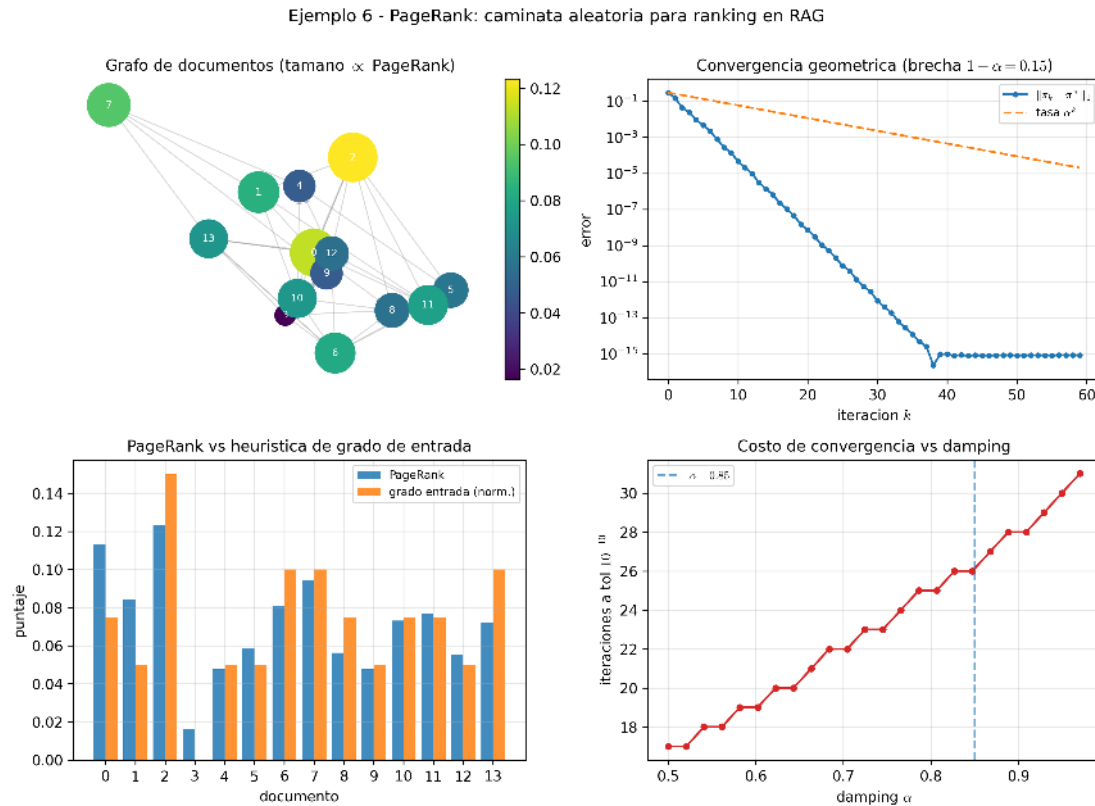


Figura 54: PageRank como caminata aleatoria para ranking en RAG. Panel 1: grafo de documentos con tamaño de nodo proporcional al PageRank. Panel 2: convergencia geométrica de la iteración de potencias y la cota de tasa α^k (Ec. 207). Panel 3: PageRank frente a la heurística de grado de entrada. Panel 4: costo de convergencia (iteraciones) frente al damping α . Resultados del Código 6 del Anexo de esta semana.

1. **PageRank no es el grado de entrada.** El documento mejor rankeado coincide en ambos criterios, pero a partir del segundo lugar difieren: el grado de entrada cuenta enlaces, mientras que PageRank pondera la importancia de quién enlaza. Un documento citado por pocos pero importantes puede superar a otro citado por muchos irrelevantes. Para un RAG, esto significa priorizar documentos centrales en la estructura del conocimiento, no solo populares.
2. **La convergencia está garantizada y es independiente del grafo.** La iteración de potencias alcanza tolerancia 10^{-15} en 40 iteraciones, y reproduce el PageRank de referencia (networkx) con error máximo 5×10^{-7} . La matriz de Google asegura $|\lambda_2| \leq \alpha = 0.85$, lo que acota la tasa de convergencia por α^k (la línea de cota del Panel 2). En este grafo el segundo autovalor observado es aún menor ($|\lambda_2| = 0.41$), de modo que la convergencia real es más rápida que la cota: α^k es una garantía de peor caso, no la tasa exacta.

3. **El damping controla el compromiso estabilidad–costo.** El Panel 4 muestra que aumentar α acerca el ranking a la estructura real de enlaces pero encarece la convergencia (la brecha espectral mínima garantizada es $1 - \alpha$, que se estrecha al crecer α). El valor estándar $\alpha = 0.85$ equilibra fidelidad al grafo y rapidez de cálculo; subirlo a 0.97 multiplica las iteraciones necesarias sin un beneficio proporcional en el ranking.
4. **El teletransporte garantiza un problema bien planteado.** El término $(1 - \alpha)^{\frac{1}{n}} \mathbf{1}\mathbf{1}^T$ vuelve la cadena irreducible y aperiódica incluso con nodos colgantes o componentes desconectadas, asegurando una estacionaria única (existencia y unicidad en el sentido de Hadamard de la Semana 1). Sin él, el ranking podría no estar definido o depender de la condición inicial.

7.16. Síntesis de la semana

Los seis ejemplos recorren el espectro de procesos markovianos, de modo que cada estudiante reconozca a cuál pertenece el fenómeno de su proyecto. La Tabla 40 resume el mapa.

Tabla 40: Mapa metodológico de la semana. Cada ejemplo ilustra un tipo distinto de proceso de Markov y la pregunta de ingeniería que resuelve.

Ej.	Tipo de proceso	Objeto clave	Pregunta resuelta
1	DTMC, 2 estados	estacionaria, recurrencia	¿qué fracción fuera de tolerancia?
2	DTMC con reparación	matriz fundamental, MTTF	¿disponibilidad y vida hasta falla?
3	CTMC, generador Q	$\exp(Qt)$, estacionaria	¿horas/año por estado, MTTF?
4	Autómata celular espacial	umbral p_c , percolación	¿incendio local o global?
5	CTMC $\rightarrow$ EDO (campo medio)	R_0 , extinción	¿brote menor o epidemia?
6	Caminata aleatoria en grafo	Perron–Frobenius, brecha	¿ranking de importancia?

Las cuatro lecciones transversales que el estudiante debe extraer son:

1. **El estado lo es todo.** La propiedad de Markov no es una limitación sino una guía de diseño: definir bien el espacio de estados es el acto modelador central. Si el modelo “recuerda” demasiado, el estado está mal definido; ampliarlo restaura la markovianidad.
2. **Dos preguntas, dos herramientas.** La distribución estacionaria responde “¿qué fracción del tiempo?” (disponibilidad, ocupación, ranking); la matriz fundamental responde “¿cuánto tarda en ocurrir?” (MTTF, tiempo a la absorción). Confundirlas produce errores de criterio, como tomar el MTTF por el MTBF.

3. **Las transiciones de fase son reales y operativas.** Percolación (Ejemplo 4) y umbral epidémico (Ejemplo 5) muestran que muchos sistemas cambian cualitativamente al cruzar un valor crítico. La gestión del riesgo se concentra en determinar de qué lado del umbral está el sistema, no en estimar parámetros lejos de él.
4. **El determinismo es un límite, no una verdad.** Los sistemas dinámicos de las semanas previas son el límite de campo medio de procesos de Markov. Cuando las poblaciones son pequeñas o los eventos raros importan, el azar reaparece y el modelo determinista deja de ser suficiente.

Apéndice Semana 7. Referencias verificables

- A1. Norris, J. R. (1997). *Markov Chains*. Cambridge University Press. <https://doi.org/10.1017/CB09780511810633>
- A2. Grinstead, C. M. y Snell, J. L. (1997). *Introduction to Probability* (2.^a ed.), cap. 11 (Markov Chains). American Mathematical Society. <https://math.dartmouth.edu/~prob/prob/prob.pdf>
- A3. Levin, D. A. y Peres, Y. (2017). *Markov Chains and Mixing Times* (2.^a ed.). American Mathematical Society. <https://doi.org/10.1090/mbk/107>
- A4. Gillespie, D. T. (1977). Exact stochastic simulation of coupled chemical reactions. *The Journal of Physical Chemistry*, 81(25), 2340–2361. <https://doi.org/10.1021/j100540a008>
- A5. Kurtz, T. G. (1970). Solutions of ordinary differential equations as limits of pure jump Markov processes. *Journal of Applied Probability*, 7(1), 49–58. <https://doi.org/10.2307/3212147>
- A6. Kermack, W. O. y McKendrick, A. G. (1927). A contribution to the mathematical theory of epidemics. *Proceedings of the Royal Society A*, 115(772), 700–721. <https://doi.org/10.1098/rspa.1927.0118>
- A7. Allen, L. J. S. (2017). A primer on stochastic epidemic models: Formulation, numerical simulation, and analysis. *Infectious Disease Modelling*, 2(2), 128–142. <https://doi.org/10.1016/j.idm.2017.03.001>
- A8. Stauffer, D. y Aharony, A. (1994). *Introduction to Percolation Theory* (2.^a ed.). Taylor & Francis. <https://doi.org/10.1201/9781315274386>
- A9. Kesten, H. (1980). The critical probability of bond percolation on the square lattice equals 1/2. *Communications in Mathematical Physics*, 74(1), 41–59. <https://doi.org/10.1007/BF01197577>
- A10. Page, L., Brin, S., Motwani, R. y Winograd, T. (1999). *The PageRank Citation Ranking: Bringing Order to the Web*. Technical Report 1999-66, Stanford InfoLab. <http://ilpubs.stanford.edu:8090/422/>
- A11. Langville, A. N. y Meyer, C. D. (2006). *Google's PageRank and Beyond: The*

Science of Search Engine Rankings. Princeton University Press. <https://doi.org/10.1515/9781400830329>

- A12. Trivedi, K. S. y Bobbio, A. (2017). *Reliability and Availability Engineering: Modeling, Analysis, and Applications*. Cambridge University Press. <https://doi.org/10.1017/9781316163047>

7.17. Anexo Semana 7. Códigos para Google Colab

Los seis códigos siguientes son autocontenidos y reproducibles: cada uno fija una semilla del generador PCG64 (`numpy.random.default_rng`), crea la carpeta Figuras/ y guarda la figura a 300 dpi. Todos los valores numéricos citados en el texto provienen de la ejecución de estos códigos. Para usarlos en Google Colab basta copiarlos en una celda y ejecutarlos; el Código 6 requiere `networkx` (preinstalado en Colab). La figura aparecerá en Figuras/ dentro del entorno.

7.17.1. Código 1 — Deriva de calibración: cadena de Markov de dos estados

Matriz de transición, distribución estacionaria por forma cerrada y por autovector, evolución $\pi_n = \pi_0 P^n$, análisis espectral (brecha y tiempo de mezcla), tiempos de primer paso y estimación de la matriz por máxima verosimilitud a partir de una trayectoria simulada.

```

1      """
2      Semana 7 - Ejemplo 1: Deriva de calibracion de un
3      sensor como cadena de
4      Markov de tiempo discreto (DTMC) de dos estados.
5
6      Estados:
7      0 = "en calibracion"    (dentro de tolerancia
8      metrologica)
9      1 = "fuera de calibracion" (excede el error maximo
10     permisible, EMP)
11
12     El tiempo avanza en pasos de inspeccion (p.ej.
13     semanas). En cada paso el
14     sensor puede derivar (0 -> 1) con probabilidad a, o
15     ser recalibrado/recuperado
16     (1 -> 0) con probabilidad b. La cadena es
17     irreducible y aperiodica, por lo que
18     posee una distribucion estacionaria unica.
19
20     Objetos de calculo:

```

```

15         - Matriz de transicion P y su potencia n-esima  $P^n$  (
Chapman-Kolmogorov).
16         - Distribucion estacionaria  $\pi$  ( $\pi P = \pi$ ).
17         - Brecha espectral  $(1 - |\lambda_2|)$  y tiempo de
mezcla.
18         - Tiempo medio de recurrencia y tiempo medio de
primer paso  $0 \rightarrow 1$ 
19         (intervalo optimo de recalibracion).
20         - Interpretacion estadistica: estimacion de P por
maxima verosimilitud
21         (MLE) a partir de una trayectoria simulada, con su
intervalo de Wald.
22         """
23
24         import numpy as np
25         import matplotlib
26         matplotlib.use('Agg') # backend sin
pantalla (Colab/headless)
27         import matplotlib.pyplot as plt
28         import matplotlib as mpl
29         import os
30
31         mpl.rcParams.update({
32             'font.size': 10, 'axes.titlesize': 11, 'axes.
labelsize': 10,
33             'figure.dpi': 110, 'savefig.dpi': 300, 'axes.
grid': True,
34             'grid.alpha': 0.3, 'mathtext.fontset': 'cm',
35         })
36         os.makedirs('Figuras', exist_ok=True)
37         rng = np.random.default_rng(20260207)
38
39         #
-----
40         # 1. Parametros del modelo (probabilidades por paso
de inspeccion)
41         #
-----
42
a = 0.08 # P(en-cal  $\rightarrow$  fuera-cal): probabilidad
de deriva por semana

```



```

43         b = 0.50          # P(fuera-cal -> en-cal): probabilidad
                             de deteccion+recalibracion
44         P = np.array([[1 - a, a],
45                        [b,      1 - b]])
46         estados = ['en calibracion', 'fuera de calibracion']
47
48         # Verificacion: cada fila es una distribucion de
         # probabilidad
49         assert np.allclose(P.sum(axis=1), 1.0), "Filas de P
         deben sumar 1"
50
51         #
         -----
52
         # 2. Distribucion estacionaria (cerrada para 2
         # estados) y por algebra
53         #
         -----
54
         # Cerrada: pi0 = b/(a+b), pi1 = a/(a+b)
55         pi_closed = np.array([b, a]) / (a + b)
56
57         # Generica: vector propio izquierdo de autovalor 1
58         w, V = np.linalg.eig(P.T)
59         idx = np.argmin(np.abs(w - 1.0))
60         pi_eig = np.real(V[:, idx])
61         pi_eig = pi_eig / pi_eig.sum()
62
63         #
         -----
64
         # 3. Evolucion transitoria pi_n = pi_0 P^n desde
         # estado "en calibracion"
65         #
         -----
66
         n_max = 40
67         pi0 = np.array([1.0, 0.0])          # arranca en
         calibracion con certeza
68         pi_n = np.zeros((n_max + 1, 2))
69         pi_n[0] = pi0
70         Pn = np.eye(2)

```

```

71         for n in range(1, n_max + 1):
72             Pn = Pn @ P
73             pi_n[n] = pi0 @ Pn
74
75             #
76             -----
77             # 4. Analisis espectral: brecha espectral y tiempo
78             de mezcla
79             #
80             -----
81
82             eigvals = np.sort(np.abs(np.linalg.eigvals(P)))
83             [::-1] # [1, /lambda2/]
84             lam2 = eigvals[1] # = /1 - a -
85             b/
86             gap = 1.0 - lam2 # brecha
87             espectral = a + b
88             # Distancia en variacion total a la estacionaria;
89             cota /lambda2|^n
90             tv = 0.5 * np.abs(pi_n - pi_closed).sum(axis=1)
91             # tiempo de mezcla empirico: primer n con TV < 0.01
92             t_mix = int(np.argmax(tv < 0.01))
93
94             #
95             -----
96
97             # 5. Tiempos caracteristicos
98             #
99             -----
100
101             # Tiempo medio de recurrencia del estado i: m_ii =
102             1/pi_i
103             m_rec = 1.0 / pi_closed
104             # Tiempo medio de primer paso 0 -> 1 (geometrico con
105             prob a): 1/a
106             T_drift = 1.0 / a # semanas
107             esperadas hasta salir de tolerancia
108             # Tiempo medio de primer paso 1 -> 0: 1/b
109             T_recover = 1.0 / b
110
111             #

```

```

97         # 6. Interpretacion estadistica: simular trayectoria
          y re-estimar P (MLE)
98         #
-----
99         def simula_cadena(P, n_pasos, s0, rng):
100             s = np.empty(n_pasos + 1, dtype=int)
101             s[0] = s0
102             for t in range(n_pasos):
103                 s[t + 1] = rng.choice(2, p=P[s[t]])
104             return s
105
106         N_sim = 5000
107         traj = simula_cadena(P, N_sim, 0, rng)
108         # conteo de transiciones n_ij
109         C = np.zeros((2, 2))
110         for t in range(N_sim):
111             C[traj[t], traj[t + 1]] += 1
112         P_mle = C / C.sum(axis=1, keepdims=True) #
estimador MLE
113         # error estandar de Wald para p_ij: sqrt(p_ij(1-p_ij
          )/n_i)
114         n_i = C.sum(axis=1)
115         se_mle = np.sqrt(P_mle * (1 - P_mle) / n_i[:, None])
116
117         #
-----
118         # 7. Salida de consola (todas las cifras citadas en
          el LaTeX)
119         #
-----
120         print("=== Ejemplo 1: deriva de calibracion (DTMC 2
          estados) ===")
121         print(f"P =\n{P}")
122         print(f"pi (cerrada) = {pi_closed}")
123         print(f"pi (autovector)= {pi_eig}")
124         print(f"Fraccion estacionaria fuera de calibracion =
          {pi_closed[1]*100:.2f} %")

```

```

125         print(f"|lambda_2| = {lam2:.4f}      brecha espectral =
        {gap:.4f}")
126         print(f"Tiempo de mezcla (TV<0.01) = {t_mix} pasos")
127         print(f"Tiempo medio recurrencia [en-cal, fuera-cal]
        = {m_rec} semanas")
128         print(f"Tiempo medio de deriva 0->1 = {T_drift:.2f}
        semanas (intervalo de recalibracion)")
129         print(f"Tiempo medio de recuperacion 1->0 = {
        T_recover:.2f} semanas")
130         print(f"P_MLE (N={N_sim}) =\n{P_mle}")
131         print(f"Error estandar Wald =\n{se_mle}")
132
133         #
        -----
134         # 8. Figura de 4 paneles
135         #
        -----

136         fig, ax = plt.subplots(2, 2, figsize=(11, 8))
137
138         # Panel 1: evolucion de  $\pi_n$  (probabilidad de cada
        estado)
139         ax[0, 0].plot(pi_n[:, 0], 'o-', ms=3, label=r'$\pi_n$
        (\mathrm{en\ cal.})$')
140         ax[0, 0].plot(pi_n[:, 1], 's-', ms=3, label=r'$\pi_n$
        (\mathrm{fuera\ cal.})$')
141         ax[0, 0].axhline(pi_closed[0], ls='--', color='C0',
        alpha=0.6)
142         ax[0, 0].axhline(pi_closed[1], ls='--', color='C1',
        alpha=0.6)
143         ax[0, 0].set_xlabel('paso $n$ (semanas)'); ax[0, 0].
        set_ylabel(r'$\pi_n$')
144         ax[0, 0].set_title(r'Evoluci\on $\pi_n=\pi_0 P^n$
        hacia la estacionaria')
145         ax[0, 0].legend(fontsize=8)
146
147         # Panel 2: convergencia a la estacionaria (TV) en
        escala log + cota  $|l_2|^n$ 
148         n_ax = np.arange(n_max + 1)
149         ax[0, 1].semilogy(n_ax, tv + 1e-16, 'o-', ms=3,
        label='TV empirica')
    
```

```

150         ax[0, 1].semilogy(n_ax, lam2**n_ax, '--', label=r'
cota  $|\lambda_2|^n$ ')
151         ax[0, 1].axhline(0.01, color='k', ls=':', alpha=0.7,
label='umbral 0.01')
152         ax[0, 1].set_xlabel('paso  $n$ '); ax[0, 1].set_ylabel
('dist. variacion total')
153         ax[0, 1].set_title(f'Mezcla geometrica ( $t_{\{mix\}}=t_{mix}$ )')
154         ax[0, 1].legend(fontsize=8)
155
156         # Panel 3: distribucion estacionaria y tiempos
caracteristicos
157         xb = np.arange(2)
158         ax[1, 0].bar(xb, pi_closed, color=['C0', 'C1'],
alpha=0.8)
159         for k, v in enumerate(pi_closed):
160             ax[1, 0].text(k, v + 0.01, f'{v:.3f}', ha='center',
fontsize=9)
161         ax[1, 0].set_xticks(xb); ax[1, 0].set_xticklabels(['
en cal.', 'fuera cal.'])
162         ax[1, 0].set_ylabel(r' $\pi_i$ '); ax[1, 0].set_ylim
(0, 1)
163         ax[1, 0].set_title(f'Estacionaria | recalibrar cada
{T_drift:.0f} sem.')
164
165         # Panel 4: sensibilidad de la fraccion fuera-cal vs
prob. de recalibracion b
166         b_vec = np.linspace(0.05, 0.95, 200)
167         pi1_vec = a / (a + b_vec)
168         ax[1, 1].plot(b_vec, pi1_vec * 100, '-')
169         ax[1, 1].axvline(b, color='C1', ls='--', alpha=0.7,
label=f' $b={b}$ ')
170         ax[1, 1].plot(b, pi_closed[1] * 100, 'o', color='C3'
, ms=7)
171         ax[1, 1].set_xlabel('prob. de recalibracion  $b$ ')
172         ax[1, 1].set_ylabel(r'% tiempo fuera de calibracion'
)
173         ax[1, 1].set_title('Sensibilidad de la
indisponibilidad metrologica')
174         ax[1, 1].legend(fontsize=8)
175
176         fig.suptitle('Ejemplo 1 - Deriva de calibracion como

```

```

177     cadena de Markov de 2 estados',
178         fontsize=12, y=1.00)
179     fig.tight_layout()
180     fig.savefig('Figuras/fig_s7_ej1_sensor_markov.png',
181         bbox_inches='tight')
182     # plt.show()
    print("\nFigura guardada: Figuras/
    fig_s7_ej1_sensor_markov.png")

```

Listing 43: Semana 7 – Código 1: Deriva de calibración (DTMC 2 estados). Ejecutar en Google Colab.

7.17.2. Código 2 — Confiabilidad de un banco de pruebas: DTMC con reparación

Distribución estacionaria y disponibilidad, matriz fundamental $N = (I - Q)^{-1}$ para el MTTF, validación Monte Carlo del tiempo a la primera falla, MTBF de régimen permanente y sensibilidad a la reparación.

```

1      """
2      Semana 7 - Ejemplo 2: Confiabilidad y disponibilidad
3      de un banco de pruebas de
4      bomba centrifuga como cadena de Markov de tiempo
5      discreto de tres estados con
6      reparacion.
7
8      Estados (paso = jornada de operacion, 1 dia):
9      O = operativo (nominal)
10     D = degradado (vibracion/cavitacion incipiente; aun
11     funcional)
12     F = falla      (fuera de servicio)
13
14     Conceptos:
15     - Distribucion estacionaria pi -> disponibilidad de
16     regimen permanente.
17     - Cadena absorbente: tiempo medio hasta la primera
18     falla (MTTF) por medio
19     de la matriz fundamental N = (I - Q)^{-1}, con Q el
20     bloque transitorio.
21     - Validacion Monte Carlo del MTTF (histograma del
22     tiempo a la absorcion).
23     - Sensibilidad de la disponibilidad frente a la tasa
24     de reparacion.

```

```

17         """
18
19         import numpy as np
20         import matplotlib
21         matplotlib.use('Agg')
22         import matplotlib.pyplot as plt
23         import matplotlib as mpl
24         import os
25
26         mpl.rcParams.update({
27             'font.size': 10, 'axes.titlesize': 11, 'axes.
28             labelsizesize': 10,
29             'figure.dpi': 110, 'savefig.dpi': 300, 'axes.
30             grid': True,
31             'grid.alpha': 0.3, 'mathtext.fontset': 'cm',
32             })
33         os.makedirs('Figuras', exist_ok=True)
34         rng = np.random.default_rng(20260208)
35
36         #
37
38         -----
39
40         # 1. Matriz de transicion (orden de estados: O, D, F
41         )
42
43         #
44
45         -----
46
47         #          ->O      ->D      ->F
48         P = np.array([
49             [0.95, 0.04, 0.01], # desde O: se mantiene, se
50             degrada o falla subito
51             [0.30, 0.65, 0.05], # desde D: se repara a O,
52             sigue degradado o falla
53             [0.50, 0.00, 0.50], # desde F: se repara a O o
54             sigue en falla
55         ])
56         nombres = ['Operativo', 'Degradado', 'Falla']
57         assert np.allclose(P.sum(axis=1), 1.0)
58
59         #
60
61         -----

```

```

47         # 2. Distribucion estacionaria -> disponibilidad
48         #
-----
49         w, V = np.linalg.eig(P.T)
50         pi = np.real(V[:, np.argmin(np.abs(w - 1.0))])
51         pi = pi / pi.sum()
52         A_full = pi[0]                                # disponibilidad
53         estricta (solo 0)
54         A_func = pi[0] + pi[1]                        # disponibilidad
55         funcional (0 o D)
56         #
-----
57         # 3. Cadena absorbente para el MTTF: F como estado
58         absorbente
59         # Transitorios T = {0, D}; bloque Q (T->T);
60         matriz fundamental N=(I-Q)^-1
61         # Tiempo esperado a la absorcion: t = N 1
62         #
-----
63         Q = P[np.ix_([0, 1], [0, 1])]                # bloque entre
64         estados transitorios
65         I2 = np.eye(2)
66         N = np.linalg.inv(I2 - Q)
67         t_abs = N @ np.ones(2)                        # [desde 0,
68         desde D]
69         MTTF_0 = t_abs[0]                             # dias
70         esperados hasta primera falla desde 0
71         #
-----
72         # Tasa de falla de regimen y MTBF aproximado (
73         frecuencia de entradas a F)
74         # flujo estacionario hacia F = sum_i pi_i P[i,F]
75         flujo_F = pi @ P[:, 2]
76         MTBF = 1.0 / flujo_F                          # dias entre
77         fallas en regimen permanente
78         #
-----

```



```
72         # 4. Evolucion transitoria desde 0 (probabilidad de
           cada estado)
73         #
           -----
74         n_max = 60
75         prob = np.zeros((n_max + 1, 3))
76         v = np.array([1.0, 0.0, 0.0])
77         prob[0] = v
78         for n in range(1, n_max + 1):
79             v = v @ P
80             prob[n] = v
81
82         #
           -----
83         # 5. Validacion Monte Carlo del MTTF (tiempo a la
           absorcion desde 0)
84         #
           -----
85         def tiempo_a_falla(P, rng, s0=0, estado_falla=2,
           t_max=100000):
86             s, t = s0, 0
87             while s != estado_falla and t < t_max:
88                 s = rng.choice(3, p=P[s])
89                 t += 1
90             return t
91
92         M = 20000
93         ttf = np.array([tiempo_a_falla(P, rng) for _ in
           range(M)])
94         MTTF_mc = ttf.mean()
95         MTTF_se = ttf.std(ddof=1) / np.sqrt(M)
96
97         #
           -----
98         # 6. Sensibilidad: disponibilidad vs probabilidad de
           reparacion D->0
99         #
           -----
```

```

100     def disponibilidad(p_D0):
101         Pm = P.copy()
102         # redistribuir fila D manteniendo P[D,F]=0.05 fijo
103         Pm[1, 0] = p_D0
104         Pm[1, 1] = 1.0 - p_D0 - 0.05
105         if Pm[1, 1] < 0:
106             return np.nan
107         w, V = np.linalg.eig(Pm.T)
108         p = np.real(V[:, np.argmin(np.abs(w - 1.0))]); p = p
/ p.sum()
109         return p[0] + p[1]
110         pD0_vec = np.linspace(0.05, 0.90, 200)
111         A_vec = np.array([disponibilidad(x) for x in pD0_vec
]])
112
113         #
-----
114         # 7. Salida de consola
115         #
-----
116         print("=== Ejemplo 2: confiabilidad banco/bomba (
DTMC 3 estados) ===")
117         print(f"pi (0,D,F) = {pi}")
118         print(f"Disponibilidad estricta (solo 0) = {A_full
*100:.2f} %")
119         print(f"Disponibilidad funcional (0 o D) = {A_func
*100:.2f} %")
120         print(f"Indisponibilidad (F) = {pi
[2]*100:.2f} %")
121         print(f"MTTF desde 0 (matriz fundamental)= {MTTF_0
:.2f} dias")
122         print(f"MTTF desde 0 (Monte Carlo M={M}) = {MTTF_mc
:.2f} +/- {MTTF_se:.2f} dias")
123         print(f"MTBF de regimen permanente = {MTBF:.2f
} dias")
124         print(f"Matriz fundamental N =\n{N}")
125
126         #
-----

```

```

127         # 8. Figura de 4 paneles
128         #
-----
129         fig, ax = plt.subplots(2, 2, figsize=(11, 8))
130         col = ['C2', 'C1', 'C3']
131
132         # Panel 1: evolucion transitoria de probabilidades
133         for k in range(3):
134             ax[0, 0].plot(prob[:, k], color=col[k], label=
nombres[k])
135             ax[0, 0].axhline(pi[k], color=col[k], ls='--', alpha
=0.5)
136             ax[0, 0].set_xlabel('jornada $n$'); ax[0, 0].
set_ylabel('probabilidad')
137             ax[0, 0].set_title('Probabilidad de estado partiendo
de Operativo')
138             ax[0, 0].legend(fontsize=8)
139
140         # Panel 2: distribucion estacionaria +
disponibilidad
141         xb = np.arange(3)
142         ax[0, 1].bar(xb, pi, color=col, alpha=0.85)
143         for k, vv in enumerate(pi):
144             ax[0, 1].text(k, vv + 0.01, f'{vv:.3f}', ha='center'
, fontsize=9)
145             ax[0, 1].set_xticks(xb); ax[0, 1].set_xticklabels(
nombres, fontsize=8)
146             ax[0, 1].set_ylabel(r'$\pi_i$')
147             ax[0, 1].set_title(f'Estacionaria | Disp. funcional
= {A_func*100:.1f}%')
148
149         # Panel 3: histograma Monte Carlo del tiempo a la
primera falla
150         ax[1, 0].hist(ttf, bins=60, density=True, color='C0'
, alpha=0.7,
151             label=f'MC ($M=\{M\}$)')
152         ax[1, 0].axvline(MTTF_0, color='C3', ls='--', lw=2,
label=f'MTTF teorico = {MTTF_0:.0f} d')
153         ax[1, 0].set_xlabel('dias hasta primera falla'); ax
[1, 0].set_ylabel('densidad')

```

```

155         ax[1, 0].set_title('Tiempo a la absorcion (cola
exponencial)')
156         ax[1, 0].legend(fontsize=8)
157
158         # Panel 4: sensibilidad de disponibilidad vs
reparacion D->0
159         ax[1, 1].plot(pD0_vec, A_vec * 100, '-')
160         ax[1, 1].axvline(P[1, 0], color='C1', ls='--', alpha
=0.7,
161         label=f'actual $P_{{D0}}={P[1,0]}$')
162         ax[1, 1].plot(P[1, 0], A_func * 100, 'o', color='C3'
, ms=7)
163         ax[1, 1].set_xlabel(r'prob. de reparacion $P_{{D\to 0}}$')
164         ax[1, 1].set_ylabel('disponibilidad funcional [%]')
165         ax[1, 1].set_title('Sensibilidad de la
disponibilidad')
166         ax[1, 1].legend(fontsize=8)
167
168         fig.suptitle('Ejemplo 2 - Confiabilidad de un banco
de pruebas: cadena de Markov con reparacion',
169         fontsize=12, y=1.00)
170         fig.tight_layout()
171         fig.savefig('Figuras/fig_s7_ej2_bomba_confiabilidad.
png', bbox_inches='tight')
172         # plt.show()
173         print("\nFigura guardada: Figuras/
fig_s7_ej2_bomba_confiabilidad.png")
174

```

Listing 44: Semana 7 – Código 2: Confiabilidad de un banco de pruebas (DTMC con reparación). Ejecutar en Google Colab.

7.17.3. Código 3 — Confiabilidad de un OPGW: cadena de tiempo continuo

Construcción de la matriz generadora Q , distribución estacionaria por $\pi Q = \mathbf{0}$, transitorio por exponencial matricial $\exp(Qt)$, MTTF por el subgenerador transitorio y horas/año por estado.

```

1         """
2         Semana 7 - Ejemplo 3: Confiabilidad de un cable OPGW
como cadena de Markov de
3         tiempo continuo (CTMC) con matriz generadora Q.

```

```

4
5      A diferencia del Ejemplo 2 (tiempo discreto), aquí
6      el tiempo es continuo y las
7      transiciones se rigen por TASAS (eventos por año).
8      Estados:
9      N = normal
10     T = sobrecarga termica (corriente elevada;
11     reversible por enfriamiento)
12     H = hielo / galloping (sobrecarga mecanica;
13     reversible)
14     F = falla (rotura/cortocircuito; requiere reparacion
15     )
16
17     Formalismo:
18     - Matriz generadora Q (filas suman 0);  $q_{ij}$  = tasa  $i \rightarrow j$  ( $i \neq j$ ),
19      $q_{ii} = -\sum_{j \neq i} q_{ij}$ .
20     - Distribucion estacionaria:  $\pi Q = 0$ ,  $\sum(\pi) = 1$ .
21     - Probabilidades transitorias:  $P(t) = \exp(Q t)$  (
22     exponencial matricial).
23     - MTTF desde N por matriz fundamental del
24     subgenerador transitorio:
25      $m = -(Q_T)^{-1} 1$ , con  $Q_T$  el bloque entre estados
26     no absorbentes.
27     - Horas/año esperadas en cada estado =  $8760 * \pi$ .
28     ""
29
30     import numpy as np
31     from scipy.linalg import expm
32     import matplotlib
33     matplotlib.use('Agg')
34     import matplotlib.pyplot as plt
35     import matplotlib as mpl
36     import os
37
38     mpl.rcParams.update({
39         'font.size': 10, 'axes.titlesize': 11, 'axes.
40         labels.size': 10,
41         'figure.dpi': 110, 'savefig.dpi': 300, 'axes.
42         grid': True,
43         'grid.alpha': 0.3, 'mathtext.fontset': 'cm',
44     })

```

```

35         os.makedirs('Figuras', exist_ok=True)
36
37         #
38         -----
39
40         # 1. Tasas de transicion (1/ano). Orden de estados:
41         N, T, H, F
42         #
43         -----
44
45         lam_NT, mu_TN, lam_TF = 2.0, 50.0, 0.5      # normal
46         <->termica, termica->falla
47         lam_NH, mu_HN, lam_HF = 1.0, 20.0, 0.3      # normal
48         <->hielo,      hielo->falla
49         mu_FN = 12.0                                #
50         reparacion falla->normal (MTTR ~ 1 mes)
51
52         Q = np.zeros((4, 4))
53         # fila N
54         Q[0, 1] = lam_NT; Q[0, 2] = lam_NH
55         # fila T
56         Q[1, 0] = mu_TN; Q[1, 3] = lam_TF
57         # fila H
58         Q[2, 0] = mu_HN; Q[2, 3] = lam_HF
59         # fila F
60         Q[3, 0] = mu_FN
61         # diagonal
62         for i in range(4):
63             Q[i, i] = -Q[i].sum()
64         nombres = ['Normal', 'Termica', 'Hielo', 'Falla']
65         assert np.allclose(Q.sum(axis=1), 0.0)
66
67         #
68         -----
69
70         # 2. Distribucion estacionaria:  $\pi Q = 0$  (sistema [
71          $Q^T$ ;  $1^T$ ]  $\pi = [0;1]$ )
72         #
73         -----
74
75         Aeq = np.vstack([Q.T, np.ones(4)])
76         beq = np.array([0, 0, 0, 0, 1.0])
    
```

```

64         pi, *_ = np.linalg.lstsq(Aeq, beq, rcond=None)
65         A_disp = pi[0] + pi[1] + pi[2]          # disponibilidad
        = todo menos F
66         U_indisp = pi[3]                        #
        indisponibilidad
67
68         #
        -----
69         # 3. Probabilidades transitorias  $P(t) = \exp(Q t)$ 
        desde N
70         #
        -----
71         t_grid = np.linspace(0, 0.5, 300)        # anos
        (~6 meses)
72         P0 = np.array([1.0, 0, 0, 0])
73         Pt = np.array([P0 @ expm(Q * t) for t in t_grid])
74
75         #
        -----
76         # 4. MTTF desde N: F absorbente. Subgenerador
        transitorio  $Q_T$  sobre  $\{N, T, H\}$ 
77         #
        -----
78         idxT = [0, 1, 2]
79         Q_T = Q[np.ix_(idxT, idxT)].copy()
80         # en la cadena absorbente, F no devuelve flujo: las
        tasas  $i \rightarrow F$  se quedan como
81         # fuga (ya estan en la diagonal de  $Q_T$  porque  $q_{ii}$ 
        incluye  $\lambda_{iF}$ )
82         m_abs = -np.linalg.solve(Q_T, np.ones(3)) # tiempos
        medios a la absorcion
83         MTTF_N = m_abs[0]                        # anos
        hasta primera falla desde N
84
85         #
        -----
86         # 5. Horas por ano en cada estado

```

```

87      #
-----
88      horas = pi * 8760.0
89
90      #
-----
91      # 6. Sensibilidad: indisponibilidad vs tasa de
reparacion mu_FN (MTTR)
92      #
-----
93      def indisp(muFN):
94          Qm = Q.copy()
95          Qm[3, 0] = muFN
96          Qm[3, 3] = -muFN
97          Am = np.vstack([Qm.T, np.ones(4)])
98          p, *_ = np.linalg.lstsq(Am, beq, rcond=None)
99          return p[3]
100      muFN_vec = np.linspace(4, 60, 200)          # 1/ano
(MTTR de 3 meses a 6 días)
101      U_vec = np.array([indisp(x) for x in muFN_vec])
102
103      #
-----
104      # 7. Salida de consola
105      #
-----
106      print("=== Ejemplo 3: OPGW como CTMC (matriz
generadora) ===")
107      print(f"Q (1/ano) =\n{Q}")
108      print(f"pi (N,T,H,F) = {pi}")
109      print(f"Disponibilidad = {A_disp*100:.4f} %")
110      print(f"Indisponibilidad (F) = {U_indisp*100:.4f} %
-> {U_indisp*8760:.2f} h/ano")
111      print(f"Horas/ano por estado [N,T,H,F] = {horas}")
112      print(f"MTTF desde N = {MTTF_N:.3f} anos ({MTTF_N
*12:.2f} meses)")
113      print(f"Fraccion de tiempo en sobrecarga termica = {

```



```

114     pi[1]*100:.3f} % ({horas[1]:.1f} h/ano)")
115
116     #
117     -----
118
119     # 8. Figura de 4 paneles
120     #
121     -----
122
123     fig, ax = plt.subplots(2, 2, figsize=(11, 8))
124     col = ['C2', 'C1', 'C0', 'C3']
125
126     # Panel 1: P(t) transitorio desde N
127     for k in range(4):
128         ax[0, 0].plot(t_grid * 12, Pt[:, k], color=col[k],
129             label=nombres[k])
130         ax[0, 0].axhline(pi[k], color=col[k], ls='--', alpha
131             =0.4)
132         ax[0, 0].set_xlabel('tiempo [meses]'); ax[0, 0].
133             set_ylabel('probabilidad')
134         ax[0, 0].set_title(r'Transitorio $P(t)=\exp(Qt)$
135             desde Normal')
136         ax[0, 0].legend(fontsize=8); ax[0, 0].set_yscale('
137             log'); ax[0, 0].set_ylim(1e-4, 1.2)
138
139     # Panel 2: distribucion estacionaria
140     xb = np.arange(4)
141     ax[0, 1].bar(xb, pi, color=col, alpha=0.85)
142     for k, vv in enumerate(pi):
143         ax[0, 1].text(k, vv * 1.15 + 1e-4, f'{vv:.4f}', ha='
144             center', fontsize=8)
145         ax[0, 1].set_xticks(xb); ax[0, 1].set_xticklabels(
146             nombres, fontsize=8)
147         ax[0, 1].set_ylabel(r'$\pi_i$'); ax[0, 1].set_yscale
148             ('log')
149         ax[0, 1].set_title(f'Estacionaria | Disponibilidad =
150             {A_disp*100:.3f}%')
151
152     # Panel 3: horas por ano en cada estado
153     ax[1, 0].bar(xb, horas, color=col, alpha=0.85)
154     for k, vv in enumerate(horas):
155         ax[1, 0].text(k, vv * 1.15 + 1, f'{vv:.0f} h', ha='

```

```

center', fontsize=8)
142         ax[1, 0].set_xticks(xb); ax[1, 0].set_xticklabels(
nombres, fontsize=8)
143         ax[1, 0].set_ylabel('horas/ano'); ax[1, 0].
set_yscale('log')
144         ax[1, 0].set_title('Tiempo esperado por estado (8760
h/ano)')
145
146         # Panel 4: indisponibilidad vs tasa de reparacion
147         ax[1, 1].plot(muFN_vec, U_vec * 8760, '-')
148         ax[1, 1].axvline(mu_FN, color='C3', ls='--', alpha
=0.7, label=f'$\mu_{\{FN\}}=\{\mu_{FN}\}/ano')
149         ax[1, 1].plot(mu_FN, U_indisp * 8760, 'o', color='C3
', ms=7)
150         ax[1, 1].set_xlabel(r'tasa de reparacion $\mu_{\{F\}to
N}\$ [1/ano]')
151         ax[1, 1].set_ylabel('indisponibilidad [h/ano]')
152         ax[1, 1].set_title('Sensibilidad a la rapidez de
reparacion')
153         ax[1, 1].legend(fontsize=8)
154
155         fig.suptitle('Ejemplo 3 - Confiabilidad de un OPGW:
cadena de Markov de tiempo continuo',
156                     fontsize=12, y=1.00)
157         fig.tight_layout()
158         fig.savefig('Figuras/fig_s7_ej3_opgw_ctmc.png',
bbox_inches='tight')
159         # plt.show()
160         print("\nFigura guardada: Figuras/
fig_s7_ej3_opgw_ctmc.png")
161

```

Listing 45: Semana 7 – Código 3: Confiabilidad de un OPGW (CTMC, matriz generadora). Ejecutar en Google Colab.

7.17.4. Código 4 — Incendio forestal: autómatas celulares probabilísticos

Simulación de la propagación por frontera (BFS estocástico), barrido en la probabilidad de propagación p , estimación empírica del umbral de percolación p_c y caracterización de la bimodalidad cerca de la criticidad.

```
"""
```

Semana 7 - Ejemplo 4: Propagacion de incendio forestal como automata celular

probabilistico (campo de Markov espacial) y su transicion de percolacion.

Modelo (variante estocastica tipo SIR sobre reticula):

- Reticula $L \times L$ de celdas; cada celda con arbol con densidad d .*

- Una celda "ardiendo" propaga el fuego a cada vecino verde (4-vecindad)*

- con probabilidad p , e inmediatamente pasa a "quemada" (no reignita).*

- El proceso es un campo de Markov: el estado futuro de una celda depende*

- solo de su estado y el de sus vecinos inmediatos.*

Para $d=1$ este proceso es equivalente a percolacion de enlaces en la red

cuadrada, con umbral critico $p_c = 0.5$. Barriendo p alrededor de p_c se observa

la transicion de fase: aparicion subita de un incendio que cruza el dominio.

Anclaje (Garcia Djeda, monitoreo por UAV): a partir de imageneria aerea se

estiman densidad d y probabilidad de propagacion p ; el modelo predice la

probabilidad de un "gran incendio" y la fraccion quemada esperada.

"""

```
import numpy as np
```

```
import matplotlib
```

```
matplotlib.use('Agg')
```

```
import matplotlib.pyplot as plt
```

```
import matplotlib as mpl
```

```
from matplotlib.colors import ListedColormap
```

```
import os
```

```
mpl.rcParams.update({
```

```
    'font.size': 10, 'axes.titlesize': 11, 'axes.
```

```

labelsizelabelsize': 10,
31         'figure.dpi': 110, 'savefig.dpi': 300, 'mathtext
.fontset': 'cm',
32     })
33     os.makedirs('Figuras', exist_ok=True)
34     rng = np.random.default_rng(20260209)
35
36     # Codigo de estados de celda
37     VACIO, VERDE, ARDIENDO, QUEMADA = 0, 1, 2, 3
38
39     def simula_incendio(L, d, p, rng, ignicion='centro')
:
40         """Una realizacion del automata. Devuelve la malla
final y si el fuego
41         alcanzo el borde (percolacion). Propagacion por
frontera (BFS estocastico)."""
42         bosque = (rng.random((L, L)) < d).astype(np.int8)
# 1=arbol, 0=vacio
43         estado = np.where(bosque == 1, VERDE, VACIO).astype(
np.int8)
44         if ignicion == 'centro':
45             focos = [(L // 2, L // 2)]
46         else: # borde superior completo
47             focos = [(0, j) for j in range(L) if estado[0, j] ==
VERDE]
48         frontera = []
49         for (i, j) in focos:
50             if estado[i, j] == VERDE:
51                 estado[i, j] = ARDIENDO
52                 frontera.append((i, j))
53                 alcanzo_borde = False
54                 while frontera:
55                     nueva = []
56                     for (i, j) in frontera:
57                         for di, dj in ((1, 0), (-1, 0), (0, 1), (0, -1)):
58                             ni, nj = i + di, j + dj
59                             if 0 <= ni < L and 0 <= nj < L and estado[ni, nj] ==
VERDE:
60                                 if rng.random() < p: # ignicion
estocastica
61                                     estado[ni, nj] = ARDIENDO
62                                     nueva.append((ni, nj))

```

```

63         if ni == 0 or ni == L - 1 or nj == 0 or nj == L - 1:
64             alcanzo_borde = True
65             estado[i, j] = QUEMADA
66             frontera = nueva
67             n_arboles = (bosque == 1).sum()
68             n_quemadas = (estado == QUEMADA).sum()
69             frac_quemada = n_quemadas / max(n_arboles, 1)
70             return estado, frac_quemada, alcanzo_borde
71
72         #
73
74         -----
75
76         # 1. Una realizacion ilustrativa (p moderada,
77         # densidad alta)
78         #
79         -----
80
81         L_demo, d_demo, p_demo = 120, 1.0, 0.55
82         estado_demo, frac_demo, _ = simula_incendio(L_demo,
83         d_demo, p_demo, rng)
84
85         #
86         -----
87
88         # 2. Barrido en p: fraccion quemada y probabilidad
89         # de percolacion
90         #
91         -----
92
93         L, d, R = 80, 1.0, 60                                # tamaño,
94         densidad, replicas por p
95         p_vals = np.linspace(0.30, 0.75, 19)
96         frac_mean = np.zeros_like(p_vals)
97         frac_std = np.zeros_like(p_vals)
98         prob_perc = np.zeros_like(p_vals)
99         frac_pc_samples = None
100        for k, p in enumerate(p_vals):
101            fr = np.empty(R); pc = np.empty(R)
102            for r in range(R):
103                _, f, b = simula_incendio(L, d, p, rng, ignicion='
centro')
104            fr[r] = f; pc[r] = b

```

```

92         frac_mean[k] = fr.mean(); frac_std[k] = fr.std()
93         prob_perc[k] = pc.mean()
94
95         # estimacion empirica de p_c: p donde P(percola)
cruza 0.5
96         i_cross = np.argmax(prob_perc >= 0.5)
97         if 0 < i_cross < len(p_vals):
98             x0, x1 = p_vals[i_cross - 1], p_vals[i_cross]
99             y0, y1 = prob_perc[i_cross - 1], prob_perc[i_cross]
100             p_c_emp = x0 + (0.5 - y0) * (x1 - x0) / (y1 - y0)
101         else:
102             p_c_emp = p_vals[i_cross]
103
104         # distribuciones de fraccion quemada en sub/super-
critico
105         def muestras_frac(p, R2=200):
106             return np.array([simula_incendio(L, d, p, rng)[1]
107                             for _ in range(R2)])
108         frac_sub = muestras_frac(0.45)
109         frac_sup = muestras_frac(0.65)
110
111         #
112         -----
113
114         # 3. Salida de consola
115         #
116         -----
117
118         print("=== Ejemplo 4: incendio forestal (automata
119         celular probabilistico) ===")
120         print(f"Realizacion demo: L={L_demo}, d={d_demo}, p
121         ={p_demo} -> fraccion quemada={frac_demo:.3f}")
122         print(f"Barrido: L={L}, d={d}, R={R} replicas/p")
123         print(f"p_c teorico (percolacion de enlaces, red
124         cuadrada) = 0.5")
125         print(f"p_c empirico (cruce P_perc=0.5)
126         = {p_c_emp:.4f}")
127         print(f"Fraccion quemada media en p=0.45 (subcritico
128         ) = {frac_sub.mean():.4f}")
129         print(f"Fraccion quemada media en p=0.65 (
130         supercritico)= {frac_sup.mean():.4f}")
    
```

```

121      #
-----
122      # 4. Figura de 4 paneles
123      #
-----

124      fig, ax = plt.subplots(2, 2, figsize=(11, 8))
125
126      # Panel 1: realizacion (mapa de estados)
127      cmap = ListedColormap(['white', 'forestgreen', '
orangered', 'dimgray'])
128      ax[0, 0].imshow(estado_demo, cmap=cmap, vmin=0, vmax
=3, interpolation='nearest')
129      ax[0, 0].set_title(f'Realizacion ($L={L_demo}$, $p={
p_demo}$): '
130      f'{frac_demo*100:.0f}% quemado')
131      ax[0, 0].set_xticks([]); ax[0, 0].set_yticks([])
132
133      # Panel 2: fraccion quemada vs p (con banda)
134      ax[0, 1].plot(p_vals, frac_mean, 'o-', ms=4)
135      ax[0, 1].fill_between(p_vals, frac_mean - frac_std,
frac_mean + frac_std,
136      alpha=0.25)
137      ax[0, 1].axvline(0.5, color='k', ls=':', alpha=0.7,
label=r'$p_c=0.5$ teorico')
138      ax[0, 1].set_xlabel('prob. de propagacion $p$')
139      ax[0, 1].set_ylabel('fraccion quemada')
140      ax[0, 1].set_title('Parametro de orden: tamano del
incendio')
141      ax[0, 1].grid(alpha=0.3); ax[0, 1].legend(fontsize
=8)
142
143      # Panel 3: probabilidad de percolacion (cruzar el
dominio)
144      ax[1, 0].plot(p_vals, prob_perc, 's-', ms=4, color='
C3')
145      ax[1, 0].axvline(p_c_emp, color='C3', ls='--', alpha
=0.7,
146      label=f'$p_c$ emp. $={p_c_emp:.3f}$')
147      ax[1, 0].axhline(0.5, color='k', ls=':', alpha=0.5)
148      ax[1, 0].set_xlabel('prob. de propagacion $p$')

```

```

149         ax[1, 0].set_ylabel('P(fuego alcanza el borde)')
150         ax[1, 0].set_title('Transicion de fase (percolacion)
151     ')
152         ax[1, 0].grid(alpha=0.3); ax[1, 0].legend(fontsize
153     =8)
154
155         # Panel 4: distribucion de fraccion quemada sub/
156     super critico
157         ax[1, 1].hist(frac_sub, bins=30, density=True, alpha
158     =0.6,
159         color='C0', label='$p=0.45$ (subcritico)')
160         ax[1, 1].hist(frac_sup, bins=30, density=True, alpha
161     =0.6,
162         color='C3', label='$p=0.65$ (supercritico)')
163         ax[1, 1].set_xlabel('fraccion quemada'); ax[1, 1].
164     set_ylabel('densidad')
165         ax[1, 1].set_title('Bimodalidad cerca del umbral')
166         ax[1, 1].grid(alpha=0.3); ax[1, 1].legend(fontsize
167     =8)
168
169         fig.suptitle('Ejemplo 4 - Incendio forestal:
170     automata celular probabilistico y percolacion',
171         fontsize=12, y=1.00)
172         fig.tight_layout()
173         fig.savefig('Figuras/fig_s7_ej4_incendio_percolacion
174     .png', bbox_inches='tight')
175         # plt.show()
176         print("\nFigura guardada: Figuras/
177     fig_s7_ej4_incendio_percolacion.png")

```

Listing 46: Semana 7 – Código 4: Incendio forestal (autómata celular y percolación). Ejecutar en Google Colab.

7.17.5. Código 5 — SIR estocástico: Gillespie y límite determinista

Algoritmo de Gillespie para el SIR estocástico, integración de la EDO determinista, distribución del tamaño final, probabilidad de brote mayor frente a la teoría de ramificación y barrido en R_0 .

```

1         """
2         Semana 7 - Ejemplo 5: Modelo epidemiologico SIR como
        cadena de Markov de tiempo

```



```

3      continuo (CTMC) y su limite determinista (sistema
dinamico de EDO).
4
5      Estados del sistema: (S, I, R) con  $S+I+R = N$ 
constante.
6      Eventos (reacciones) y sus tasas (propensiones):
7      - Infeccion:  $S + I \rightarrow 2I$  con tasa  $\beta * S * I /$ 
N
8      - Recuperacion:  $I \rightarrow R$  con tasa  $\gamma * I$ 
9      Numero reproductivo basico:  $R_0 = \beta / \gamma$ .
10
11     Se comparan dos descripciones del mismo proceso:
12     1. Estocastica exacta (algoritmo de Gillespie / SSA)
: captura extincion
13     temprana, variabilidad y tamano final aleatorio.
14     2. Determinista (EDO de campo medio): limite  $N \rightarrow$ 
inf; es el "sistema
15     dinamico" continuo subyacente.
16
17     Resultados clave:
18     - Probabilidad de brote mayor vs menor (extincion
estocastica).
19     - Distribucion del tamano final (bimodal para  $R_0 >$ 
1).
20     - Comparacion de la prob. de brote mayor con la
teoria 1 -  $(1/R_0)^{I_0}$ .
21     ""
22
23     import numpy as np
24     from scipy.integrate import solve_ivp
25     import matplotlib
26     matplotlib.use('Agg')
27     import matplotlib.pyplot as plt
28     import matplotlib as mpl
29     import os
30
31     mpl.rcParams.update({
32         'font.size': 10, 'axes.titlesize': 11, 'axes.
labelsize': 10,
33         'figure.dpi': 110, 'savefig.dpi': 300, 'axes.
grid': True,
34         'grid.alpha': 0.3, 'mathtext.fontset': 'cm',

```

```

35         })
36         os.makedirs('Figuras', exist_ok=True)
37         rng = np.random.default_rng(20260210)
38
39         #
40
41         -----
42
43         # 1. Parametros
44         #
45         -----
46
47         N = 1000                # poblacion total
48         I0 = 5                  # infectados iniciales
49         beta = 0.60             # tasa de contacto efectivo
50         [1/dia]
51         gamma = 0.25            # tasa de recuperacion [1/dia]
52         ]
53
54         R0 = beta / gamma        # numero reproductivo basico
55         print("=== Ejemplo 5: SIR estocastico (CTMC) vs
56         determinista (EDO) ===")
57         print(f"N={N}, I0={I0}, beta={beta}, gamma={gamma},
58         R0={R0:.2f}")
59
60         #
61         -----
62
63         # 2. Gillespie SSA (una realizacion)
64         #
65         -----
66
67         def gillespie_sir(N, I0, beta, gamma, rng, t_max
68         =200.0):
69             S, I, R = N - I0, I0, 0
70             t = 0.0
71             ts, Is, Ss = [t], [I], [S]
72             while I > 0 and t < t_max:
73                 a_inf = beta * S * I / N                # propension de
74                 infeccion
75                 a_rec = gamma * I                       # propension de
76                 recuperacion
77                 a0 = a_inf + a_rec
78                 if a0 <= 0:

```

```

62         break
63         t += rng.exponential(1.0 / a0)  # tiempo al
        siguiente evento ~ Exp(a0)
64         if rng.random() < a_inf / a0:  # cual evento ocurre
65             S -= 1; I += 1
66         else:
67             I -= 1; R += 1
68             ts.append(t); Is.append(I); Ss.append(S)
69             tamano_final = N - S  # total de
        individuos que se infectaron
70         return np.array(ts), np.array(Ss), np.array(Is),
        tamano_final
71
72         #
        -----
73         # 3. EDO determinista (campo medio)
74         #
        -----
75         def sir_ode(t, y):
76             S, I, R = y
77             dS = -beta * S * I / N
78             dI = beta * S * I / N - gamma * I
79             dR = gamma * I
80             return [dS, dI, dR]
81             sol = solve_ivp(sir_ode, [0, 120], [N - I0, I0, 0],
82             t_eval=np.linspace(0, 120, 600), rtol=1e-8, atol=1e
-8)
83             I_peak_det = sol.y[1].max()
84             t_peak_det = sol.t[np.argmax(sol.y[1])]
85             final_det = N - sol.y[0, -1]
86
87             #
            -----
88             # 4. Ensemble estocastico: distribucion del tamano
            final
89             #
            -----
90
            M = 3000

```

```

91         tamanos = np.empty(M)
92         trayectorias = []
93         for m in range(M):
94             ts, Ss, Is, tf = gillespie_sir(N, I0, beta, gamma,
rng)
95             tamanos[m] = tf
96             if m < 30:                                     # guardar
algunas para graficar
97                 trayectorias.append((ts, Is))
98
99             # clasificacion brote menor vs mayor (umbral en 5%
de la poblacion)
100             umbral = 0.05 * N
101             p_mayor_emp = (tamanos > umbral).mean()
102             p_mayor_teo = 1.0 - (1.0 / R0) ** I0          # prob. de
brote mayor (proc. de ramificacion)
103             tam_mayores = tamanos[tamanos > umbral]
104             print(f"Pico determinista I_max = {I_peak_det:.1f}
en t={t_peak_det:.1f} d")
105             print(f"Tamano final determinista = {final_det:.1f}
({final_det/N*100:.1f}% de N)")
106             print(f"P(brote mayor) empirica = {p_mayor_emp:.3f}"
)
107             print(f"P(brote mayor) teorica 1-(1/R0)^I0 = {
p_mayor_teo:.3f}")
108             print(f"Tamano final medio (brotes mayores) = {
tam_mayores.mean():.1f} "
109                   f"({tam_mayores.mean()/N*100:.1f}% de N)")
110
111             #
-----

112             # 5. Barrido de R0: prob. de brote mayor vs teoria
113             #
-----

114             R0_vec = np.linspace(0.6, 3.0, 13)
115             p_mayor_vec = np.empty_like(R0_vec)
116             for k, R0k in enumerate(R0_vec):
117                 bk = R0k * gamma
118                 cnt = 0; Mk = 800
119                 for _ in range(Mk):

```

```

120         _, _, _, tf = gillespie_sir(N, I0, bk, gamma, rng)
121         cnt += (tf > umbral)
122         p_mayor_vec[k] = cnt / Mk
123         p_teo_vec = np.where(R0_vec > 1, 1.0 - (1.0 / np.
maximum(R0_vec, 1e-9)) ** I0, 0.0)
124
125         #
-----
126         # 6. Figura de 4 paneles
127         #
-----
128         fig, ax = plt.subplots(2, 2, figsize=(11, 8))
129
130         # Panel 1: trayectorias estocasticas I(t) +
determinista
131         for (ts, Is) in trayectorias:
132             ax[0, 0].plot(ts, Is, color='C0', alpha=0.25, lw
=0.8)
133             ax[0, 0].plot(sol.t, sol.y[1], 'C3', lw=2.5, label='
EDO determinista')
134             ax[0, 0].set_xlabel('tiempo [dias]'); ax[0, 0].
set_ylabel('infectados $I(t)$')
135             ax[0, 0].set_title(f'Trayectorias (CTMC) vs campo
medio ($R_0={R0:.1f}$)')
136             ax[0, 0].legend(fontsize=8); ax[0, 0].set_xlim(0,
120)
137
138         # Panel 2: distribucion del tamaño final (bimodal)
139         ax[0, 1].hist(tamamos, bins=50, density=True, color=
'C0', alpha=0.75)
140         ax[0, 1].axvline(final_det, color='C3', ls='--', lw
=2,
141             label=f'final EDO = {final_det:.0f}')
142         ax[0, 1].set_xlabel('tamaño final del brote'); ax[0,
1].set_ylabel('densidad')
143         ax[0, 1].set_title(f'Bimodalidad: extincion vs brote
mayor (M={M})')
144         ax[0, 1].legend(fontsize=8)
145
146         # Panel 3: retrato de fase S-I (sistema dinamico)

```

```

147         ax[1, 0].plot(sol.y[0], sol.y[1], 'C3', lw=2.5,
label='EDO')
148         for (ts, Is) in trayectorias[:15]:
149             # reconstruir S a partir de la trayectoria no es
directo; graficar I vs t-rango
150             pass
151             ax[1, 0].axvline(N / R0, color='k', ls=':', alpha
=0.7,
152             label=r'$S^*=N/R_0$ (umbral)')
153             ax[1, 0].set_xlabel('susceptibles $$$'); ax[1, 0].
set_ylabel('infectados $I$')
154             ax[1, 0].set_title('Retrato de fase del sistema
dinamico')
155             ax[1, 0].legend(fontsize=8)
156
157             # Panel 4: prob. de brote mayor vs R0
158             ax[1, 1].plot(R0_vec, p_mayor_vec, 'o-', ms=5, label
='SSA (empirica)')
159             ax[1, 1].plot(R0_vec, p_teo_vec, '--', color='C3',
label=r'$1-(1/R_0)^{\{I_0\}}$')
160             ax[1, 1].axvline(1.0, color='k', ls=':', alpha=0.6,
label='$R_0=1$')
161             ax[1, 1].set_xlabel('$R_0$'); ax[1, 1].set_ylabel('P
(brote mayor)')
162             ax[1, 1].set_title('Umbral epidemico y extincion
estocastica')
163             ax[1, 1].legend(fontsize=8)
164
165             fig.suptitle('Ejemplo 5 - SIR estocastico (Gillespie
) y su limite determinista',
166             fontsize=12, y=1.00)
167             fig.tight_layout()
168             fig.savefig('Figuras/fig_s7_ej5_sir_estocastico.png'
, bbox_inches='tight')
169             # plt.show()
170             print("\nFigura guardada: Figuras/
fig_s7_ej5_sir_estocastico.png")
171
172

```

Listing 47: Semana 7 – Código 5: SIR estocástico (Gillespie) y EDO determinista. Ejecutar en Google Colab.

7.17.6. Código 6 — PageRank: caminata aleatoria para ranking en RAG

Construcción de la matriz de Google con corrección de nodos colgantes, PageRank por iteración de potencias con registro de convergencia, verificación contra networkx, comparación con el grado de entrada y sensibilidad al damping.

```

1      """
2          Semana 7 - Ejemplo 6: Ranking por caminata aleatoria
          (PageRank) para
3          recuperacion de documentos en un sistema RAG.
4
5          Un sistema RAG (Retrieval-Augmented Generation)
          ordena documentos por
6          relevancia. Cuando los documentos forman un grafo
          dirigido (citas, enlaces o
7          similitud semantica umbralizada), su importancia
          puede medirse con la
8          distribucion estacionaria de una caminata aleatoria:
          PageRank.
9
10         Cadena de Markov del "navegante aleatorio":
11         - Con probabilidad alpha (damping) sigue un enlace
          saliente uniforme.
12         - Con probabilidad (1-alpha) "teletransporta" a un
          documento al azar.
13         - Nodos sin enlaces (dangling) se tratan como
          teletransporte total.
14         Matriz de Google:  $G = \alpha * (S) + (1-\alpha) * (1/n) * \mathbf{1} \mathbf{1}^T$ ,
15         con S la matriz de transicion estocastica por filas
          (incluye correccion de
16         nodos colgantes). El vector PageRank es el
          autovector izquierdo de G con
17         autovalor 1 (Perron-Frobenius), calculado por
          iteracion de potencias.
18
19         La brecha espectral de G es  $1 - \alpha$ , lo que fija
          la tasa de convergencia
20          $\sim \alpha^k$  de la iteracion de potencias (
          independiente del grafo).
21         """
22
23         import numpy as np

```

```

24     import networkx as nx
25     import matplotlib
26     matplotlib.use('Agg')
27     import matplotlib.pyplot as plt
28     import matplotlib as mpl
29     import os
30
31     mpl.rcParams.update({
32         'font.size': 10, 'axes.titlesize': 11, 'axes.
33         labelsizes': 10,
34         'figure.dpi': 110, 'savefig.dpi': 300, 'mathtext
35         .fontset': 'cm',
36         })
37     os.makedirs('Figuras', exist_ok=True)
38     rng = np.random.default_rng(20260211)
39
40     #
41     -----
42
43     # 1. Grafo dirigido de documentos (n nodos). Aristas
44     # i->j: "i referencia a j"
45     #
46     -----
47
48     n = 14
49     G = nx.gnp_random_graph(n, 0.22, seed=42, directed=
50     True)
51     # garantizar al menos un nodo colgante (sin salida)
52     # para ilustrar la correccion
53     G.remove_edges_from(list(G.out_edges(7)))
54     A = nx.to_numpy_array(G, nodelist=range(n)) # A[
55     i,j]=1 si i->j
56
57     #
58     -----
59
60     # 2. Matriz de transicion S con correccion de nodos
61     # colgantes
62     #
63     -----
64
65     out_deg = A.sum(axis=1)

```



```

51         S = np.zeros((n, n))
52         for i in range(n):
53             if out_deg[i] > 0:
54                 S[i] = A[i] / out_deg[i]                # enlace
55             else:
56                 S[i] = np.ones(n) / n                    # dangling
57             alpha = 0.85
58             G_mat = alpha * S + (1 - alpha) * np.ones((n, n)) /
59             n
60             assert np.allclose(G_mat.sum(axis=1), 1.0)
61             #
62             -----
63             # 3. PageRank por iteracion de potencias (
64             registrando convergencia)
65             #
66             -----
67
68             pi = np.ones(n) / n
69             hist = []
70             pi_exact = None
71             for k in range(100):
72                 pi_new = pi @ G_mat
73                 hist.append(np.abs(pi_new - pi).sum())    # ||pi_
74                 {k+1}-pi_k||_1
75                 if hist[-1] < 1e-15:
76                     pi_exact = pi_new; break
77                 pi = pi_new
78                 pr = pi / pi.sum()
79                 # distancia al limite para graficar tasa alpha^k
80                 pr_ref = pr.copy()
81                 pi2 = np.ones(n) / n
82                 dist_lim = []
83                 for k in range(60):
84                     dist_lim.append(np.abs(pi2 - pr_ref).sum())
85                 pi2 = pi2 @ G_mat
86
87             # verificacion contra networkx
88             pr_nx = np.array([nx.pagerank(G, alpha=alpha)[i] for

```

```

84         i in range(n)])
85
86         #
87         -----
88
89         # 4. Comparacion con grado de entrada (heuristica
90         ingenua)
91         #
92         -----
93
94         in_deg = A.sum(axis=0)
95         in_deg_norm = in_deg / in_deg.sum() if in_deg.sum()
96         > 0 else in_deg
97         orden_pr = np.argsort(pr)[::-1]
98         orden_in = np.argsort(in_deg)[::-1]
99
100        # brecha espectral teorica y observada
101        gap_teo = 1 - alpha
102        # segundo autovalor en magnitud
103        ev = np.sort(np.abs(np.linalg.eigvals(G_mat.T)))
104        [::-1]
105        lam2 = ev[1]
106
107        print("=== Ejemplo 6: PageRank (caminata aleatoria)
108        para RAG ===")
109        print(f"n={n} documentos, alpha={alpha}, nodos
110        colgantes={int((out_deg==0).sum())}")
111        print(f"PageRank (power iter) top-5 nodos: {orden_pr
112        [:5]}")
113        print(f"Ranking por grado de entrada top-5: {
114        orden_in[:5]}")
115        print(f"Max |PageRank - networkx| = {np.abs(pr -
116        pr_nx).max():.2e}")
117        print(f"Iteraciones a tol 1e-15 = {len(hist)}")
118        print(f"|lambda_2(G)| = {lam2:.4f}      brecha
119        espectral = {1-lam2:.4f}      (teorica 1-alpha={gap_teo:.2f})")
120        print(f"PageRank nodo top = {orden_pr[0]} con score
121        {pr[orden_pr[0]]:.4f}")
122        print(f"Coincide top-1 PageRank vs grado entrada? {
123        orden_pr[0]==orden_in[0]}")
124
125        #
    
```

```

-----
110         # 5. Sensibilidad: iteraciones a convergencia vs
alpha
111         #
-----

112         def iters_converge(alpha_, tol=1e-10):
113             Gm = alpha_ * S + (1 - alpha_) * np.ones((n, n)) / n
114             p = np.ones(n) / n
115             for k in range(2000):
116                 pn = p @ Gm
117                 if np.abs(pn - p).sum() < tol:
118                     return k + 1
119                 p = pn
120             return 2000
121             alpha_vec = np.linspace(0.50, 0.97, 24)
122             iters_vec = np.array([iters_converge(a) for a in
alpha_vec])
123
124         #
-----

125         # 6. Figura de 4 paneles
126         #
-----

127         fig, ax = plt.subplots(2, 2, figsize=(11, 8))
128
129         # Panel 1: grafo con tamaño de nodo ~ PageRank
130         pos = nx.spring_layout(G, seed=7)
131         nx.draw_networkx_edges(G, pos, ax=ax[0, 0], alpha
=0.3, arrowsize=8,
132             edge_color='gray', width=0.8)
133         nodes = nx.draw_networkx_nodes(G, pos, ax=ax[0, 0],
node_size=pr * 9000 + 60,
134             node_color=pr, cmap='viridis')
135         nx.draw_networkx_labels(G, pos, ax=ax[0, 0],
font_size=7, font_color='white')
136         ax[0, 0].set_title('Grafo de documentos (tamaño $\\
propto$ PageRank)')
137         ax[0, 0].axis('off')

```

```

138         plt.colorbar(nodes, ax=ax[0, 0], fraction=0.046, pad
139         =0.04)
140
141         # Panel 2: convergencia de la iteracion de potencias
142         (escala log)
143         kk = np.arange(len(dist_lim))
144         ax[0, 1].semilogy(kk, np.array(dist_lim) + 1e-18, 'o
145         -', ms=3,
146         label=r'$\\pi_k - \\pi^{*}_1$')
147         ax[0, 1].semilogy(kk, alpha ** kk * dist_lim[0], '--
148         ',
149         label=r'tasa $\\alpha^k$')
150         ax[0, 1].set_xlabel('iteracion $k$'); ax[0, 1].
151         set_ylabel('error')
152         ax[0, 1].set_title(f'Convergencia geometrica (brecha
153         $1 - \\alpha = {1 - alpha:.2f}$)')
154         ax[0, 1].grid(alpha=0.3); ax[0, 1].legend(fontsize
155         =8)
156
157         # Panel 3: PageRank vs grado de entrada normalizado
158         x = np.arange(n); width = 0.4
159         ax[1, 0].bar(x - width / 2, pr, width, label='
160         PageRank', color='C0', alpha=0.85)
161         ax[1, 0].bar(x + width / 2, in_deg_norm, width,
162         label='grado entrada (norm.)',
163         color='C1', alpha=0.85)
164         ax[1, 0].set_xlabel('documento'); ax[1, 0].
165         set_ylabel('puntaje')
166         ax[1, 0].set_title('PageRank vs heuristica de grado
167         de entrada')
168         ax[1, 0].set_xticks(x); ax[1, 0].grid(alpha=0.3,
169         axis='y')
170         ax[1, 0].legend(fontsize=8)
171
172         # Panel 4: iteraciones a convergencia vs damping
173         alpha
174         ax[1, 1].plot(alpha_vec, iters_vec, 'o-', ms=4,
175         color='C3')
176         ax[1, 1].axvline(alpha, color='C0', ls='--', alpha
177         =0.7, label=f'$\\alpha = {alpha}$')
178         ax[1, 1].set_xlabel(r'damping $\\alpha$')
179         ax[1, 1].set_ylabel('iteraciones a tol $10^{-10}$')
    
```

```
165         ax[1, 1].set_title('Costo de convergencia vs damping  
'  
166     ')  
167         ax[1, 1].grid(alpha=0.3); ax[1, 1].legend(fontsize  
168     =8)  
169         fig.suptitle('Ejemplo 6 - PageRank: caminata  
170     aleatoria para ranking en RAG',  
171     fontsize=12, y=1.00)  
172         fig.tight_layout()  
173         fig.savefig('Figuras/fig_s7_ej6_pagerank_rag.png',  
174     bbox_inches='tight')  
175         # plt.show()  
176         print("\nFigura guardada: Figuras/  
fig_s7_ej6_pagerank_rag.png")
```

Listing 48: Semana 7 – Código 6: PageRank (caminata aleatoria) para RAG. Ejecutar en Google Colab.

7.18. Ejercicios Reto — Tarea Semana 7

Los Ejercicios 1 y 2 son obligatorios. El Ejercicio 3 otorga punto extra. Todos deben anclarse al *proyecto doctoral propio* del estudiante: el espacio de estados, las transiciones y los datos provienen de su propia investigación.

7.18.1. Ejercicio 1 (obligatorio) — Modelo de Markov del propio proyecto

Identifique en su proyecto un fenómeno con evolución markoviana (deriva, degradación, disponibilidad, ocupación, propagación o ranking) y defina su espacio de estados. Entregue: (i) la justificación de la propiedad de Markov y de la homogeneidad temporal, declarando explícitamente qué información codifica el estado; (ii) la matriz de transición P (tiempo discreto) o la matriz generadora Q (tiempo continuo), con cada transición motivada físicamente; (iii) la distribución estacionaria resuelta por (195) o (204), con su interpretación (disponibilidad, fracción de tiempo, ranking); y (iv) la clasificación de la cadena (irreducible/aperiódica, estados absorbentes si los hay).

Rúbrica.

- 3 Espacio de estados bien definido, propiedad de Markov justificada, P o Q correcta y motivada, estacionaria resuelta e interpretada, clasificación correcta.
- 2 Modelo correcto pero con propiedad de Markov no justificada, o estacionaria sin interpretación física.

- 1 Matriz mal normalizada (filas que no suman 1, o Q con filas que no suman 0) o estados mal definidos.
- 0 Sin modelo markoviano del proyecto.

7.18.2. Ejercicio 2 (obligatorio) — Tiempos característicos y validación por simulación

Para el modelo del Ejercicio 1, calcule un tiempo característico relevante: tiempo medio de recurrencia (197), tiempo medio hasta la absorción por la matriz fundamental (200)/(205) (MTTF), o tiempo de mezcla por la brecha espectral (207). Implemente en Google Colab una simulación reproducible (`numpy.random.default_rng(semilla)`) del proceso y valide el resultado analítico contra el promedio de al menos $M \geq 10^4$ trayectorias, reportando el error estándar. Discuta el acuerdo (o desacuerdo) y su origen.

Rúbrica.

- 3 Tiempo característico correcto por vía analítica, simulación reproducible, validación con error estándar y discusión del acuerdo.
- 2 Cálculo analítico correcto pero validación sin error estándar, o simulación no reproducible.
- 1 Simulación que corre pero sin contraste cuantitativo con la teoría.
- 0 Sin cálculo de tiempos característicos ni validación.

7.18.3. Ejercicio 3 — Transición de fase o propagación de incertidumbre

Elija una de las dos vías. **(a) Transición de fase:** si su fenómeno admite una representación espacial o de propagación, identifique un parámetro de control, estime su umbral crítico por barrido y caracterice el parámetro de orden y su variabilidad cerca de la criticidad. **(b) Propagación de incertidumbre:** estime su matriz de transición a partir de datos (reales o sintéticos) por máxima verosimilitud (212), propague la incertidumbre de las entradas $\hat{p}_{ij}$ a la distribución estacionaria o al MTTF mediante Monte Carlo (técnica de la Semana 6) y reporte el intervalo de cobertura del 95 % de la cantidad de interés.

Rúbrica.

- 3 (a) Umbral estimado y parámetro de orden caracterizado con su variabilidad; o (b) MLE correcto, propagación Monte Carlo e intervalo de cobertura bien construido.
- 0 No entregado o sin caracterización cuantitativa del umbral / del intervalo.